

Performance Analysis and Optimization of CNN Training from Scratch Using CUDA

Shiva Zare

Moein Yeganeh

February 15, 2026

Abstract

This project presents a complete implementation of a Convolutional Neural Network (CNN) training pipeline built entirely in CUDA from scratch. We implement a LeNet-style architecture for MNIST digit classification, developing all forward and backward propagation kernels without relying on high-level frameworks like PyTorch or TensorFlow. Our work focuses on understanding GPU architecture fundamentals, implementing four distinct optimization techniques (shared memory tiling, kernel fusion, optimized backward pass with atomic reduction, and stream-based data transfer overlapping), and conducting performance analysis using NVIDIA profiling tools. The implementation demonstrates how CNN workloads map onto GPU hardware, identifies computational bottlenecks, and achieves significant speedup through systematic optimization.

Contents

1	Introduction	5
1.1	Objectives	5
1.2	Dataset: MNIST	5
2	Background	5
2.1	Convolutional Neural Networks	5
2.2	Backpropagation Algorithm	6
2.3	GPU Architecture Fundamentals	7
3	System Setup	7
3.1	Hardware Requirements	7
3.2	Software Requirements	8
3.3	Dataset Preparation	9
3.4	Project Compilation	9
4	Project Architecture	10
4.1	Directory Structure	10
4.2	Data Structures	11
4.3	Network Architecture Details	11
5	Methodology	12
5.1	Network Architecture Design	12
5.1.1	Architecture Selection Rationale	12
5.1.2	Layer Configuration	12
5.2	Implementation Approach	12
5.2.1	Development Phases	12
5.3	Memory Management Strategy	13
5.3.1	Memory Allocation Philosophy	13
5.3.2	Buffer Management	13
5.4	Training Pipeline	13
5.4.1	Hyperparameters	15
5.5	Design	15
5.5.1	Baseline Configuration	15
5.5.2	Convolutional Layer	15

5.5.3	Max Pooling Layer	16
5.5.4	Fully Connected Layer	16
5.5.5	ReLU Activation	17
5.5.6	Softmax + Cross-Entropy Loss	17
5.5.7	SGD Optimizer	18
5.6	Profiling:	18
6	Optimization Techniques	21
6.1	Optimized Configuration	21
6.2	Optimization 1: Shared Memory Tiling for Convolution	22
6.2.1	Motivation and Problem	22
6.2.2	Solution: Cooperative Loading into Shared Memory	22
6.2.3	Why This Works	24
6.2.4	Implementation Details	24
6.2.5	Expected Performance Impact	25
6.3	Profiling:	25
6.4	Optimization 2: Kernel Fusion (Conv + Bias + ReLU)	28
6.4.1	Problem: Intermediate Memory Operations	28
6.4.2	Solution: Fused Kernel	29
6.4.3	Benefits	29
6.4.4	Expected Performance Impact	29
6.5	Profiling:	30
6.6	Optimization 3: Optimized Backward Pass with Atomic Reduction	36
6.6.1	Problem: Atomic Contention	36
6.6.2	Solution: Two-Level Reduction	37
6.6.3	Performance Improvement	38
6.7	Optimization 4: Stream-Based Data Transfer Overlapping	38
6.7.1	Problem: CPU-GPU Transfer Latency	38
6.7.2	Solution: Concurrent Execution with Streams	39
6.8	Profiling:	39
7	Performance Analysis and Results	46
7.1	Test Parameters	46
7.1.1	Profiling Tools Usage	46
7.2	Profiling Summary Table	46

7.2.1	Definition of GPU Active Time	46
7.3	Analysis of Results	47
7.3.1	Baseline Performance Characteristics	47
7.3.2	Optimization 1: Tiled Convolution Impact	48
7.3.3	Optimization 2: Kernel Fusion Impact	48
7.3.4	Tile Size Experiment (8×8 vs 16×16)	49
7.3.5	Optimization 3+4: Dramatic Improvement	49
7.3.6	Breaking Down Optimization 3+4 Contributions	50
8	Conclusions	51
8.1	Project Summary	51
8.1.1	About Neural Network Training	51
8.1.2	About GPU Programming	51
9	References	52

1 Introduction

1.1 Objectives

Modern deep learning frameworks like PyTorch and TensorFlow have revolutionized machine learning by abstracting away implementation complexity. However, this abstraction creates a “black box” that obscures the fundamental operations occurring on GPU hardware.

The reason that we use convolution layers for image is that same filter detects features anywhere in the image , it has fewer parameters than fully connected networks , and early layers detect simple features (edges), later layers detect complex patterns.

Primary Objectives:

1. Implement a complete CNN architecture (LeNet-style) entirely in CUDA
2. Develop all forward and backward propagation kernels from scratch
3. Apply and measure distinct GPU optimization techniques
4. Achieve functional training on the MNIST handwritten digit dataset
5. Conduct systematic performance analysis using profiling tools

1.2 Dataset: MNIST

The MNIST (Modified National Institute of Standards and Technology) database is a widely-used benchmark for image classification:

- **Training Set:** 60,000 images (28×28 grayscale)
- **Test Set:** 10,000 images (28×28 grayscale)
- **Labels:** 10 classes (digits 0-9)
- **Format:** IDX binary file format
- **Preprocessing:** Pixel normalization ($0-255 \rightarrow 0.0-1.0$)

2 Background

2.1 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a class of deep learning models specifically designed for processing grid-like data such as images. Unlike fully connected networks, CNNs exploit spatial structure through:

Forward Pass : Compute predictions given inputs

Input $\rightarrow$ Conv1 $\rightarrow$ ReLU $\rightarrow$ Pool1 $\rightarrow$ Conv2 $\rightarrow$ ReLU $\rightarrow$ Pool2 $\rightarrow$ FC1 $\rightarrow$ ReLU $\rightarrow$ FC2 $\rightarrow$ Softmax

Backward Pass : Compute gradients using chain rule

Loss gradient $\rightarrow$ FC2 backward $\rightarrow$ FC1 backward $\rightarrow$ Pool2 backward $\rightarrow$ Conv2 backward $\rightarrow$ Pool1 backward $\rightarrow$ Conv1 backward

Key Components:

1. **Convolutional Layers:** Apply learnable filters across the image to detect features. The convolution operation for output position (n, c_{out}, h, w) is:

$$Y[n, c_{out}, h, w] = b[c_{out}] + \sum_{c_{in}=0}^{C_{in}-1} \sum_{k_h=0}^{K-1} \sum_{k_w=0}^{K-1} X[n, c_{in}, h \cdot s - p + k_h, w \cdot s - p + k_w] \cdot W[c_{out}, c_{in}, k_h, k_w] \quad (1)$$

where s is stride, p is padding, and K is kernel size.

2. **Pooling Layers:** Reduce spatial dimensions while preserving important features. Max pooling computes:

$$Y[n, c, h, w] = \max_{i,j \in \text{window}} X[n, c, h \cdot s + i, w \cdot s + j] \quad (2)$$

3. **Activation Functions:** Introduce non-linearity. ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x) \quad (3)$$

4. **Fully Connected Layers:** Combine features for final classification:

$$Y = XW^T + b \quad (4)$$

2.2 Backpropagation Algorithm

Training neural networks requires computing gradients of the loss function with respect to all parameters. The backpropagation algorithm applies the chain rule:

Forward Pass: Compute predictions given inputs

$$\hat{y} = f(x; \theta) \quad (5)$$

Backward Pass: Compute gradients using chain rule

$$\frac{\partial L}{\partial \theta_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial \theta_i} \quad (6)$$

Weight Update: Adjust parameters to minimize loss (SGD)

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L \quad (7)$$

where η is the learning rate.

Each layer must: Store inputs during forward pass (needed for backward pass), Compute three gradients during backward:

To propagate to previous layer

$$\frac{\partial L}{\partial x} \quad (8)$$

To update parameters

$$\frac{\partial L}{\partial W} \quad (9)$$

To update biases

$$\frac{\partial L}{\partial Bias} \quad (10)$$

Store inputs during forward pass (needed for backward pass) Compute three gradients during backward:

2.3 GPU Architecture Fundamentals

Modern GPUs are designed for massive parallelism, fundamentally different from CPUs:

GPU Memory Hierarchy:

- **Registers:** Fastest, per-thread storage (32-bit values)
- **Shared Memory:** Fast, per-block storage , enables thread cooperation in a block
- **L1/L2 Cache:** Automatic caching of global memory
- **Global Memory:** Largest but slowest
- **Local Memory:** Located in Global memory

CUDA Execution Model:

- **Thread:** Single execution unit
- **Warp:** Group of 32 threads executing together
- **Block:** Group of threads sharing memory
- **Grid:** Collection of blocks launched in a kernel

3 System Setup

3.1 Hardware Requirements

Our Test System:

- **GPU:** NVIDIA GPU with Compute Capability ≥ 6.0 (Pascal or newer)
- **Compute Capability:** sm_61 (Pascal architecture)

3.2 Software Requirements

Required Software:

- **CUDA Toolkit:** Version 12.4 or higher
- **Build System:** Make or Python 3
- **Profiling Tools:** Nsight Systems and Nsight Compute

Installation Steps:

```

1 # Check GPU availability
2 nvidia-smi
3
4 # Install CUDA Toolkit
5 sudo apt install cuda-toolkit-12-4
6
7 # Verify installation
8 nvcc --version
9
10 # Verify profiling tools
11 nsys --version
12 ncu --version

```

Listing 1: GPU Driver Verification

Tue Feb 10 20:39:42 2026

NVIDIA-SMI 581.80				Driver Version: 581.80				CUDA Version: 13.0			
GPU	Name	Perf	Driver-Model	Bus-Id	Disp.A	Memory-Usage	Volatile	Uncorr.	ECC		
Fan	Temp		Pwr:Usage/Cap				GPU-Util	Compute M.			
0	NVIDIA GeForce RTX 4070	...	WDDM	00000000:01:00.0	On				N/A		
N/A	37C	P8	2W / 140W	1207MiB /	8188MiB		0%	Default	N/A		
Processes:											
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage					
ID	ID	ID									
0	N/A	N/A	6316	C+G	...Microsoft VS Code\Code.exe	N/A					
0	N/A	N/A	6872	C+G	...NearbyShare\nearby_share.exe	N/A					
0	N/A	N/A	10912	C+G	C:\Windows\explorer.exe	N/A					
0	N/A	N/A	11016	C+G	...0.3719.115\msedgeview2.exe	N/A					
0	N/A	N/A	11328	C+G	...2txyewy\CrossDeviceResume.exe	N/A					
0	N/A	N/A	11744	C+G	...ows\System32\NahimicSvc64.exe	N/A					
0	N/A	N/A	11784	C+G	...xyewy\ShellExperienceHost.exe	N/A					
0	N/A	N/A	13208	C+G	...em_tray\lghub_system_tray.exe	N/A					
0	N/A	N/A	13268	C+G	...cw5n1h2txyewy\SearchHost.exe	N/A					
0	N/A	N/A	13284	C+G	...y\StartMenuExperienceHost.exe	N/A					
0	N/A	N/A	14144	C+G	...t\Edge\Application\msedge.exe	N/A					
0	N/A	N/A	16900	C+G	...8bbwe\PhoneExperienceHost.exe	N/A					
0	N/A	N/A	17944	C+G	...5n1h2txyewy\TextInputHost.exe	N/A					
0	N/A	N/A	18820	C+G	...0_x64__8j3eq9eme6cct\IGCC.exe	N/A					
0	N/A	N/A	19404	C+G	...indows\System32\ShellHost.exe	N/A					
0	N/A	N/A	19568	C+G	...yb3d8bbwe\WindowsTerminal.exe	N/A					
0	N/A	N/A	19780	C+G	...ing\Adobe\Connect\Connect.exe	N/A					
0	N/A	N/A	21724	C+G	...1.115\msedgeview2.exe	N/A					

Figure 1: GPU Driver Information


```
=== GPU Information ===  
GPU Name: NVIDIA GeForce RTX 4070 Laptop GPU  
Compute Capability: 8.9  
Warp Size: 32  
Shared Memory per Block: 49152 bytes  
Shared Memory Banks: 32  
=====
```

Figure 2: GPU Information

```
nvcc: NVIDIA (R) Cuda compiler driver  
Copyright (c) 2005-2025 NVIDIA Corporation  
Built on Fri_Nov__7_19:25:04_Pacific_Standard_Time_2025  
Cuda compilation tools, release 13.1, V13.1.80  
Build cuda_13.1.r13.1/compiler.36836380_0
```

Figure 3: CUDA Toolkit Version

```
NVIDIA (R) Nsight Compute Command Line Profiler  
Copyright (c) 2018-2025 NVIDIA Corporation  
Version 2025.4.0.0 (build 36690805) (public-release)
```

Figure 4: Nsight Compute Version

3.3 Dataset Preparation

```
1 # Download dataset  
2 wget http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz  
3 wget http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz  
4 wget http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz  
5 wget http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz  
6  
7 # Extract  
8 gunzip train-images-idx3-ubyte.gz  
9 gunzip train-labels-idx1-ubyte.gz  
10 gunzip t10k-images-idx3-ubyte.gz  
11 gunzip t10k-labels-idx1-ubyte.gz
```

Listing 2: MNIST Dataset Download and Extraction

3.4 Project Compilation

```
1 # Using Makefile  
2 make  
3  
4 # Or using Python build script  
5 python3 build.py
```

```

6
7 # Run training
8 ./cnn --train-images train-images.idx3-ubyte \
9       --train-labels train-labels.idx1-ubyte \
10      --test-images t10k-images.idx3-ubyte \
11      --test-labels t10k-labels.idx1-ubyte \
12      --epochs 5 --batch 64 --lr 0.01 --block 256
13
14 # Remove the build files
15 make clean

```

Listing 3: Building the Project

4 Project Architecture

4.1 Directory Structure

```

1 project/
2     include/                                # Header files
3         activations.h                       # ReLU declarations
4         conv_layer.h                       # Convolution interface
5         conv_tile.h                        # Tiled convolution
6         conv_fused.h                      # Fused Conv+Bias+ReLU
7         conv_backward.h                   # Optimized backward
8         fc_layer.h                        # Fully connected interface
9         pool_layer.h                     # Max pooling interface
10        loss.h                            # Softmax + cross-entropy
11        optimizer.h                       # SGD update
12        tensor.h                          # Tensor data structure
13        timer.h                           # CUDA event timing
14        utils.h                           # Error checking
15        mnist_loader.h                    # Dataset loading
16
17    src/                                    # Source files
18        main.cu                           # Entry point
19        train_op.cu                        # Training loop
20        mnist_loader.cpp                   # MNIST parser
21        layers/                            # Core implementations
22            fc_forward.cu
23            fc_backward.cu
24            pool_forward.cu
25            pool_backward.cu
26        ops/                              # Operations
27            relu_activate.cu
28            sgd_update.cu
29            softmax_loss.cu
30        optimize/                          # Optimized versions
31            conv_tile.cu
32            conv_fuse.cu
33            conv_back_tile.cu
34        utils/
35            cuda_utils.cu
36            timer.cu

```

37		
38	Makefile	# Build configuration
39	build.py	# Python build script

Listing 4: Project Directory Structure

4.2 Data Structures

Tensor Structure:

```

1 struct Tensor {
2     float* data;    // Device pointer to GPU memory
3     int N;          // Batch size
4     int C;          // Number of channels
5     int H;          // Height
6     int W;          // Width
7 };

```

Listing 5: Tensor Data Structure (NCHW Layout)

In modern frameworks like Pytorch transform their input data into tensors and then use them for training and testing models , so we implement and use Tensor structures here to store information about each batch of data .The NCHW (Batch, Channel, Height, Width) layout is chosen for coalesced memory access in convolution kernels

4.3 Network Architecture Details

Layer	Input Shape	Output Shape	Parameters
Input	(B, 1, 28, 28)	(B, 1, 28, 28)	-
Conv1	(B, 1, 28, 28)	(B, 8, 28, 28)	W: $8 \times 1 \times 3 \times 3 = 72$, b: 8
ReLU1	(B, 8, 28, 28)	(B, 8, 28, 28)	-
MaxPool1	(B, 8, 28, 28)	(B, 8, 14, 14)	-
Conv2	(B, 8, 14, 14)	(B, 16, 14, 14)	W: $16 \times 8 \times 3 \times 3 = 1152$, b: 16
ReLU2	(B, 16, 14, 14)	(B, 16, 14, 14)	-
MaxPool2	(B, 16, 14, 14)	(B, 16, 7, 7)	-
Flatten	(B, 16, 7, 7)	(B, 784)	-
FC1	(B, 784)	(B, 128)	W: $128 \times 784 = 100352$, b: 128
ReLU3	(B, 128)	(B, 128)	-
FC2	(B, 128)	(B, 10)	W: $10 \times 128 = 1280$, b: 10
Softmax	(B, 10)	(B, 10)	-
Total			103,018 parameters

Table 1: Network Architecture Layer-by-Layer Breakdown

5 Methodology

5.1 Network Architecture Design

5.1.1 Architecture Selection Rationale

We selected a LeNet-style architecture. This architecture is chosen for several reasons:

1. **Proven Effectiveness:** Well-established for MNIST (95-99% accuracy)
2. **Manageable Complexity:** Simple enough to implement from scratch while demonstrating core CNN concepts

5.1.2 Layer Configuration

Convolutional Layers: Apply learnable filters across the image to detect features (edges, textures, patterns).

- **Kernel Size 3×3 :** Standard choice providing good balance between receptive field and computational cost
- **Padding 1:** Preventing information loss at borders
- **Channel Progression ($1\rightarrow 8\rightarrow 16$):** Gradually increases feature map complexity

Pooling Layers: Reduce spatial dimensions while preserving important features

- **2×2 Max Pooling:** Standard downsampling reducing dimensions by $2\times$
- **Stride 2:** Non-overlapping windows for efficient computation

Fully Connected Layers: Combine features to make final predictions

- **Hidden Layer (128 units)**
- **Output Layer (10 units):** One per class (digits 0-9)

5.2 Implementation Approach

5.2.1 Development Phases

Phase 1: Baseline Implementation

1. Implement naive kernels for all layers and verify mathematical correctness
2. Achieve training convergence on MNIST and good accuracy

We profile baseline implementation , identify computational bottlenecks , implement distinct optimization techniques , and measure performance improvements for each optimization .

Phase 2: Optimization

1. Shared-memory tiled convolution to reuse input tiles and reduce global memory traffic
2. Fuse Conv + Bias + ReLU in the forward pass to reduce intermediate global reads/writes and reduce launch overhead.
3. reduce atomic contention using block-level partial partial sum followed by a merge/reduction step.

5.3 Memory Management Strategy

5.3.1 Memory Allocation Philosophy

All GPU memory is allocated once at initialization and reused throughout training to minimize allocation overhead.

5.3.2 Buffer Management

Forward Pass Activations: All intermediate activations must be saved for backward pass:

- d_R1 : Conv1+ReLU output (needed for ReLU backward)
- d_P1 : Pool1 output (needed for Conv2 input)
- $d_argmax1$: Pool1 max positions (needed for Pool1 backward)

Gradient Buffers: Separate gradient buffers for each layer:

- d_dw1, d_dw2 : Weight gradients
- d_db1, d_db2 : Bias gradients
- d_dR1, d_dR2 : Activation gradients

Ping-Pong Buffers for Stream Overlapping: Enables overlap of data transfer with computation

- $d_X[2]$: Two input buffers (alternate between batches)
- $h_Xpin[2]$: Two pinned host buffers (for async H2D transfer)

5.4 Training Pipeline

Algorithm 1 Training Pipeline Data Flow

```
1: Initialization:
2: Load MNIST dataset into CPU memory
3: Allocate all GPU buffers (weights, activations, gradients)
4: Initialize weights with He initialization
5: Create CUDA streams and events
6: Allocate pinned host memory for async transfers
7:
8: for epoch = 1 to num_epochs do
9:   Shuffle dataset indices
10:  for step = 1 to num_steps do
11:    Async Data Transfer:
12:    Prepare next batch on CPU (normalize, format)
13:    Transfer next batch to GPU asynchronously (stream overlap)
14:
15:    Forward Pass:
16:    Conv1 + ReLU (fused kernel)
17:    MaxPool1
18:    Conv2 + ReLU (fused kernel)
19:    MaxPool2
20:    FC1 + ReLU
21:    FC2
22:    Softmax + Cross-Entropy Loss
23:
24:    Backward Pass:
25:    Zero gradient buffers
26:    FC2 backward (compute  $dW_{fc2}$ ,  $db_{fc2}$ ,  $dR3$ )
27:    ReLU3 backward (mask gradient)
28:    FC1 backward (compute  $dW_{fc1}$ ,  $db_{fc1}$ ,  $dP2$ )
29:    Pool2 backward (route gradient to max positions)
30:    ReLU2 backward (mask gradient)
31:    Conv2 backward (compute  $dW_2$ ,  $db_2$ ,  $dP1$ )
32:    Pool1 backward
33:    ReLU1 backward
34:    Conv1 backward (compute  $dW_1$ ,  $db_1$ )
35:
36:    Optimizer:
37:    SGD update:  $W \leftarrow W - \eta \nabla W$ 
38:
39:    Logging:
40:    if step mod 100 == 0 then
41:      Copy loss and logits to CPU
42:      Compute accuracy
43:      Print metrics
44:    end if
45:  end for
46:  Evaluate on test set
47:  Print epoch summary
48: end for
```

5.4.1 Hyperparameters

Hyperparameter	Value
Batch Size	
Learning Rate	0.01
Epochs	5-10
Optimizer	SGD (no momentum)
Weight Initialization	He Normal
Loss Function	Cross-Entropy

Table 2: Training Hyperparameters

5.5 Design

5.5.1 Baseline Configuration

To measure optimization impact, we establish a baseline:

- Naive convolution (no shared memory)
- Separate Conv, Bias, ReLU kernels (no fusion)
- Naive backward pass (high atomic contention)
- Sequential H2D transfers (no stream overlapping)

5.5.2 Convolutional Layer

1. Forward Pass:

$$Y[n, c_{out}, h, w] = b[c_{out}] + \sum_{c_{in}=0}^{C_{in}-1} \sum_{k_h=0}^{K-1} \sum_{k_w=0}^{K-1} X[n, c_{in}, h \cdot s - p + k_h, w \cdot s - p + k_w] \cdot W[c_{out}, c_{in}, k_h, k_w] \quad (11)$$

In naive forward convolution there is one thread per output pixel, each thread reads $K \times K \times C$ values from global memory which means there is no reuse and same input pixels read multiple times by different threads.

2. Backward Pass - Three Gradients:

1. Input Gradient:

$$\frac{\partial L}{\partial X[n, c_{in}, h, w]} = \sum_{c_{out}} \sum_{k_h} \sum_{k_w} \frac{\partial L}{\partial Y[n, c_{out}, h_{out}, w_{out}]} \cdot W[c_{out}, c_{in}, k_h, k_w] \quad (12)$$

2. Weight Gradient:

$$\frac{\partial L}{\partial W[c_{out}, c_{in}, k_h, k_w]} = \sum_n \sum_h \sum_w \frac{\partial L}{\partial Y[n, c_{out}, h, w]} \cdot X[n, c_{in}, h \cdot s - p + k_h, w \cdot s - p + k_w] \quad (13)$$

3. *Bias Gradient:*

$$\frac{\partial L}{\partial b[c_{out}]} = \sum_n \sum_h \sum_w \frac{\partial L}{\partial Y[n, c_{out}, h, w]} \quad (14)$$

5.5.3 Max Pooling Layer

In forward max pooling we store argmax the reason is that during backward pass, gradient only flows through the maximum position, all other positions in the pool window get zero gradient, so storing argmax avoids recomputing max during backward.

We use **atomicAdd** because multiple output positions might max-pool from overlapping windows

1. **Forward Pass:**

$$Y[n, c, h, w] = \max_{i \in [0, P-1], j \in [0, P-1]} X[n, c, h \cdot s + i, w \cdot s + j] \quad (15)$$

Save argmax indices during forward pass:

$$\text{argmax}[n, c, h, w] = \arg \max_{i, j} X[n, c, h \cdot s + i, w \cdot s + j] \quad (16)$$

2. **Backward Pass:**

$$\frac{\partial L}{\partial X[n, c, h, w]} = \begin{cases} \frac{\partial L}{\partial Y[n, c, h_{pool}, w_{pool}]} & \text{if } (h, w) \text{ was max} \\ 0 & \text{otherwise} \end{cases} \quad (17)$$

5.5.4 Fully Connected Layer

Forward Pass:

$$Y = XW^T + b \quad \text{or} \quad Y[n, o] = b[o] + \sum_{i=0}^{D_{in}-1} X[n, i] \cdot W[o, i] \quad (18)$$

Backward Pass - Three Gradients:

1. *Input Gradient:*

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \cdot W \quad \text{or} \quad \frac{\partial L}{\partial X[n, i]} = \sum_o \frac{\partial L}{\partial Y[n, o]} \cdot W[o, i] \quad (19)$$

2. *Weight Gradient:*

$$\frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial Y} \right)^T \cdot X \quad \text{or} \quad \frac{\partial L}{\partial W[o, i]} = \sum_n \frac{\partial L}{\partial Y[n, o]} \cdot X[n, i] \quad (20)$$

3. *Bias Gradient:*

$$\frac{\partial L}{\partial b[o]} = \sum_n \frac{\partial L}{\partial Y[n, o]} \quad (21)$$

5.5.5 ReLU Activation

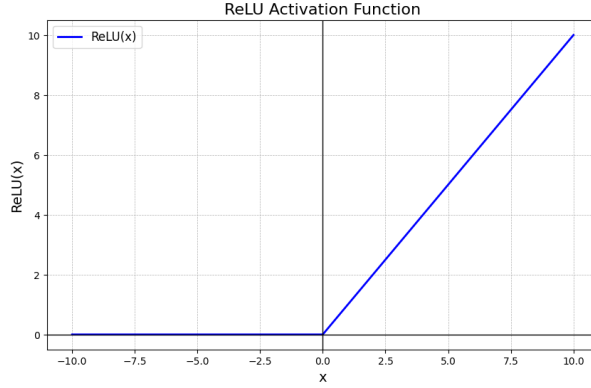


Figure 5: ReLU Activation

Forward Pass:

$$Y[i] = \max(0, X[i]) = \begin{cases} X[i] & \text{if } X[i] > 0 \\ 0 & \text{if } X[i] \leq 0 \end{cases} \quad (22)$$

Backward Pass:

$$\frac{\partial L}{\partial X[i]} = \begin{cases} \frac{\partial L}{\partial Y[i]} & \text{if } X[i] > 0 \\ 0 & \text{if } X[i] \leq 0 \end{cases} \quad (23)$$

5.5.6 Softmax + Cross-Entropy Loss

Softmax (Forward): Converts logits to probabilities

$$p_i = \frac{e^{z_i - \max(z)}}{\sum_{j=1}^K e^{z_j - \max(z)}} \quad (\text{numerically stable}) \quad (24)$$

Cross-Entropy Loss:

$$L = -\frac{1}{N} \sum_{n=1}^N \log(p_{n,y_n}) \quad (25)$$

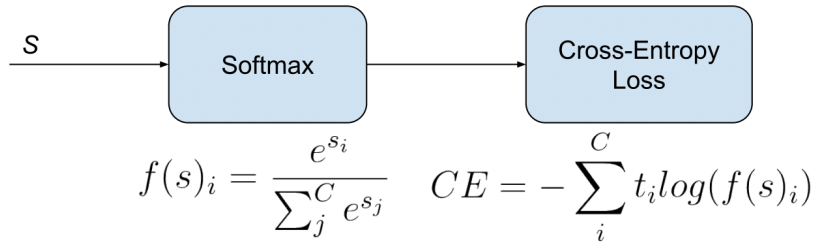


Figure 6: Softmax Cross Entropy

5.5.7 SGD Optimizer

Update Rule:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L \quad (26)$$

For each parameter:

$$W[i, j] \leftarrow W[i, j] - \eta \cdot \frac{\partial L}{\partial W[i, j]} \quad (27)$$

where η is the learning rate (e.g., 0.01).

5.6 Profiling:

In this section we will profile different configuration for block size and batch size that may effect our performance :

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 64 --lr 0.01 --
  block 256
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
82.0	4971520640	2811	1448424.3	8211.0	2354	7235495	2053545.0	cudaDeviceSynchronize
6.8	339490619	37010	9172.9	6827.0	2440	6830575	36779.1	cudaLaunchKernel
5.2	259286860	4068	63738.2	28871.5	9148	579476	68337.3	cudaMemcpy
4.5	222355063	40	5558876.6	1811.0	1041	221788081	35052522.4	cudaMalloc
1.4	69248893	8433	8211.7	5779.0	2191	101548	7427.2	cudaMemset
0.1	3748920	37010	101.3	71.0	48	41848	438.4	cudaKernelGetName
0.0	870800	40	21952.0	1731.5	956	273313	51484.4	cudaFree
0.0	332120	10	33212.0	30401.0	20525	61753	12471.5	cuLibraryLoadData
0.0	259641	10	25964.1	11650.5	7383	138284	40092.2	cuLibraryUnload
0.0	10634	1	10634.0	10634.0	10634	10634	0.0	cuModuleGetLoadingMode
0.0	3981	16	248.8	225.5	143	509	91.8	cuLibraryGetKernel
0.0	2713	1	2713.0	2713.0	2713	2713	0.0	cuCtxSynchronize
0.0	311	1	311.0	311.0	311	311	0.0	cuDeviceGetUuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
 [libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\protobuf\wire_data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
 [6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
73.7	19172763	2038	9407.6	16672.0	288	32416	8417.4	[CUDA memcpy Host-to-Device]
19.2	4997136	8433	592.6	352.0	288	2016	331.9	[CUDA memset]
7.0	1828201	2030	900.6	896.0	767	1632	93.1	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
386.115	8433	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.021	2038	0.108	0.201	0.000	0.401	0.100	[CUDA memcpy Host-to-Device]
2.802	2030	0.001	0.003	0.000	0.003	0.001	[CUDA memcpy Device-to-Host]

Figure 7: Batch_S : 64, Block_S : 256

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
65.6	2756021502	1874	147082.7	1789163.5	622297	4287373	845338.4	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int)
27	1132900365	1874	604535.9	798071	369467	1996008	243045.7	conv_db_kernel(const float *, float *, int, int, int, int)
2.9	121102399	1874	64622.4	95422.5	29471	603257	40657.7	conv_dh_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int)
1.5	640101996	2186	29609.7	31111.5	7007	79407	22570.6	conv_fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
1.2	52097970	2186	23832.6	28735.5	14112	51359	9612.6	conv_forward_naive_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
0.3	13567300	1874	7239.8	7856	2527	18016	4658.6	conv_backward_weight_kernel(const float *, const float *, float *, int, int, int, int)
0.3	13261948	1874	7087.5	7312	1472	21279	5577.9	conv_backward_input_kernel(const float *, const float *, float *, int, int, int, int)
0.2	8150880	7496	1087.4	960	895	5016	432.3	sgd_update_kernel(float *, float *, int, float, int)
0.2	8030423	2911	2947.2	2956	1055	12224	1630.3	relu_backward_kernel(const float *, const float *, float *, int)
0.2	7478461	3279	2280.7	2272	927	8640	1083.9	relu_forward_kernel(const float *, float *, int)
0.2	6329562	2186	2895.5	3328	2239	6976	647.1	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int, int, int)
0.1	5267688	1874	2810.9	3040	2240	6048	494.9	maxpool_backward_kernel(const float *, const float *, const float *, float *, int, int, int, int, int, int, int, int)
0.1	4187411	1874	2234.5	2624	1696	4000	477.9	zero_kernel(float *, int)
0.1	3293147	1874	1757.3	1792	1600	2816	146.6	conv_backward_bias_kernel(const float *, float *, int, int)
0	1833067	937	1956.3	1920	1887	4096	160.9	softmax_cross_entropy_loss_grad_kernel(const float *, const float *, float *, float *, int, int)
0	897526	937	957.9	960	927	1408	63	scale_inplace_kernel(float *, int, float)

Figure 8: Batch_S : 64, Block_S : 256

1. GPU Active Time: 4199899 us

2. GPU Util : 84.5%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 32 --lr 0.01 --
  block 256
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
80.1	4068835336	5625	723348.5	7818.0	2206	3564651	1022234.7	cudaDeviceSynchronize
8.1	408943411	74058	5535.4	3733.0	2388	3129637	13649.8	cudaLaunchKernel
7.1	358677076	8132	44106.9	16658.5	5535	566214	53006.6	cudaMemcpy
3.2	160907329	40	4022683.2	1513.0	1062	160510780	25377396.3	cudaMalloc
1.5	76788840	16875	4550.4	3793.0	2360	164241	4136.6	cudaMemset
0.1	5026460	74058	67.9	57.0	47	30851	283.6	cuKernelGetName
0.0	871678	40	21792.0	1545.0	1053	265944	61280.2	cudaFree
0.0	347649	10	34764.9	31179.0	20334	63235	12492.9	cuLibraryLoadData
0.0	232824	10	23282.4	10303.0	7067	125512	36393.8	cuLibraryUnload
0.0	8731	1	8731.0	8731.0	8731	8731	0.0	cuModuleGetLoadingMode
0.0	3077	16	192.3	193.0	102	342	71.0	cuLibraryGetKernel
0.0	2787	1	2787.0	2787.0	2787	2787	0.0	cuCtxSynchronize
0.0	271	1	271.0	271.0	271	271	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dev\p4\build\sw\devtools\Agora\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3.21.1\src\google\proto
ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
60.5	21416536	4070	5262.0	8992.0	288	32607	4625.2	[CUDA memcpy Host-to-Device]
29.7	10506248	16875	622.6	416.0	287	1856	332.4	[CUDA memset]
9.9	3488224	4062	858.7	864.0	767	1632	55.4	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
772.643	16875	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.122	4070	0.054	0.100	0.000	0.401	0.050	[CUDA memcpy Host-to-Device]
2.807	4062	0.001	0.001	0.000	0.001	0.001	[CUDA memcpy Device-to-Host]

Figure 9: Batch_S : 32, Block_S : 256

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
63.2	268062715	3750	714896.7	945445	311900	2035304	405706.3	copy_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int)
26.5	1122894744	3750	299438.6	336796	184830	943797	116374.8	copy_db_kernel(const float *, float *, int, int, int, int, int, int, int, int, int, int, int)
3.4	143805008	3750	38348.2	55775.5	16928	508346	30076.7	copy_dx_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int)
3.1	129987560	4374	29718.2	30992	6975	80063	22794.3	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
1.4	57791442	4374	13212.5	14943.5	7776	26127	5377	copy_forward_naive_kernel(Tensor, Tensor, const float *, float *, int, int, int, int, int, int, int, int)
0.5	19382291	3750	5168.6	5408	1407	12160	3733.7	backward_input_kernel(const float *, const float *, float *, int, int, int)
0.4	15884184	15000	1058.9	928	895	3040	299.2	sgd_update_kernel(float *, float *, int, float, int)
0.4	15804302	3750	4214.5	4544	1759	9856	2446.8	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.3	10679457	5625	1898.6	1792	1023	4128	729.4	relu_backward_kernel(const float *, const float *, float *, int)
0.2	10514589	6561	1602.6	1568	895	3456	559.5	relu_forward_kernel(const float *, float *, int)
0.2	9130175	4374	2097.4	2304	1695	3552	373	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int, int, int)
0.2	7523525	3750	2006.3	2144	1631	3296	284.9	maxpool_backward_kernel(const float *, const int *, float *, int, int, int, int, int, int, int, int, int, int)
0.1	5756060	3750	1534.9	1696	1247	2624	272.4	zero_kernel(float *, int)
0.1	5017514	3750	1338	1344	1247	2112	142.7	fc_backward_bias_kernel(const float *, float *, int, int)
0.1	3584279	1875	1911.6	1856	1855	2912	196.7	softmax_cg_loss_grad_kernel(const float *, const int *, float *, float *, int, int)
0	1787467	1875	953.3	928	896	1536	93.5	scale_inplace_kernel(float *, int, float)

Figure 10: Batch_S : 32, Block_S : 256

1. GPU Active Time: 4498256 us

2. GPU Util : 88.5%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
  1024
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
47.0	4241844882	22500	192970.9	7695.0	2119	1459259	263691.9	cudaDeviceSynchronize
25.5	2354836758	32500	72414.1	14706.5	3333	1000490	96076.6	cudaMemcpy
21.6	1992486187	296250	6725.4	3711.0	2451	4460226	12391.4	cudaLaunchKernel
4.5	412393321	67500	6109.5	3832.0	2194	375511	7351.3	cudaMemset
1.1	104889692	40	2622242.3	1236.0	926	104726511	16558058.9	cudaMalloc
0.3	25021003	296250	84.5	59.0	48	53958	509.2	cuKernelGetName
0.0	613652	40	15341.3	1330.5	774	374182	64063.4	cudaFree
0.0	346714	10	34671.4	30433.5	21401	58511	13246.1	cuLibraryLoadData
0.0	276914	10	27691.4	11502.5	7167	112715	33369.2	cuLibraryUnload
0.0	8648	1	8648.0	8648.0	8648	8648	0.0	cuModuleGetLoadingMode
0.0	4080	1	4080.0	4080.0	4080	4080	0.0	cuCtxSynchronize
0.0	3648	16	228.0	226.0	110	485	97.3	cuLibraryGetKernel
0.0	312	1	312.0	312.0	312	312	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\devs\pa\build\sw\devtools\aggr\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3.21.1\src\google\proto] ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
51.5	41194861	67500	610.3	384.0	287	13792	327.5	[CUDA memset]
31.1	24924967	16258	1533.1	2336.0	287	32576	1180.2	[CUDA memcpy Host-to-Device]
17.4	13944066	16250	858.1	864.0	767	14496	121.4	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.172	16258	0.014	0.025	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
2.830	16250	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 11: $Batch_S : 8, Block_S : 1024$

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
44.2	2922407519	15000	194827.2	221021.5	109631	838422	87456.3	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
25.8	1710074770	17500	97718.6	97486.5	3615	294332	94395.1	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int, int)
16.9	1121025021	15000	74735	85007.5	46943	565049	31208.3	conv_db_kernel(const float *, float *, int, int, int, int)
6.2	410238250	15000	27349.2	32111.5	18432	54880	8622.6	conv_dk_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
1.6	102064971	17500	5863.1	7951.5	3072	11808	1905.7	conv_forward_naive_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.1	71805280	15000	4787	5168	1599	13376	3167.8	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
1	68868045	60000	1147.8	960	927	7104	418.7	sgd_update_kernel(float *, float *, int, float, int)
0.6	41357523	15000	2757.2	3120	1535	5792	1194.1	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.5	33045651	17500	1886.3	1888	1823	6432	112.6	maxpool_forward_kernel(const float *, float *, int, int, int, int, int, int, int, int)
0.5	30300489	26250	1154.3	1024	991	2176	200.0	relu_forward_kernel(const float *, float *, int)
0.4	29695893	22500	1319.8	1152	1119	4928	251.3	relu_backward_kernel(const float *, const float *, float *, int)
0.4	24555905	15000	1637.1	1632	1567	4704	105.1	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int)
0.2	16145999	15000	1076.4	1050	1023	4608	75.2	fc_backward_bias_kernel(const float *, float *, int, int)
0.2	15216897	15000	1014.5	1120	831	1728	150.7	zero_kernel(float *, int)
0.2	13785111	7500	1830	1824	1728	5824	135.6	softmax_cross_entropy_loss_grad_kernel(const float *, const float *, float *, float *, int, int)
0.1	7257301	7500	967.6	960	927	3072	64.4	scale_inplace_kernel(float *, int, float)

Figure 12: $Batch_S : 8, Block_S : 1024$

1. GPU Active Time: 6611119 us
2. GPU Util : 71.6%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-ubyte
--train-labels train-labels.idx1-ubyte --test-images t10k-images.idx3-ubyte
--test-labels t10k-labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
256
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
50.1	4867011644	22500	180756.1	28533.5	2464	1404171	242495.5	cudaDeviceSynchronize
27.6	2239987582	296250	7561.1	3765.0	2499	2363705	10793.6	cudaLaunchKernel
13.9	1126742263	32508	34608.5	15957.0	4237	3219045	38324.5	cudaMemcpy
6.4	516627043	67500	7653.7	3937.0	2189	135192	8493.6	cudaMemset
1.8	148395923	40	3709898.1	1275.0	962	148315376	23434180.2	cudaMalloc
0.3	24801610	296250	83.7	57.0	47	128148	673.9	cuKernelGetName
0.0	513695	40	12842.4	1371.5	773	321539	53628.1	cudaFree
0.0	446555	10	44655.5	46766.0	22533	60488	10817.9	cuLibraryLoadData
0.0	258635	10	25863.5	10265.0	7267	109455	31954.6	cuLibraryUnload
0.0	8144	1	8144.0	8144.0	8144	8144	0.0	cuModuleGetLoadingMode
0.0	3684	16	230.3	219.0	149	372	56.9	cuLibraryGetKernel
0.0	3108	1	3108.0	3108.0	3108	3108	0.0	cuCtxSynchronize
0.0	288	1	288.0	288.0	288	288	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3.21.1\src\google\proto] data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
51.6	41073320	67500	608.5	352.0	287	3200	336.0	[CUDA memset]
31.0	24638222	16258	1515.5	2304.0	287	32319	1161.7	[CUDA memcpy Host-to-Device]
17.4	13846097	16250	852.1	832.0	768	1696	48.5	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.172	16258	0.014	0.000	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
2.830	16250	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 13: $Batch_S : 8, Block_S : 256$

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
53.7	2658692390	15000	177246.2	193821.5	78495	883605	100532.8	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
22.6	1120775761	15000	74718.4	85119	46975	500922	28513.0	conv_db_kernel(const float *, float *, int, int, int, int)
9.8	484444780	17500	27682.6	28079.5	3584	79743	24052.6	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
6	29891599	15000	19872.8	26048	12863	367035	8892.9	conv_dx_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
2	97175403	17500	5552.9	6399.5	3168	11680	2304.7	conv_forward_naive_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.3	64156276	60000	1069.3	960	895	2816	290.1	sgd_update_kernel(float *, float *, int, float, int)
0.9	42310646	15000	2820.7	4095	1407	5984	1394.1	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
0.7	32389105	15000	2159.3	2400	1183	8064	961.8	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.6	27861670	26250	1061.4	1056	895	1792	118.4	relu_forward_kernel(const float *, float *, int)
0.5	27048274	17500	1545.6	1536	1439	2464	119.5	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.5	26618555	22500	1183	1184	1023	1920	127	relu_backward_kernel(const float *, const float *, float *, int)
0.4	20149117	15000	1343.3	1376	1215	2080	115.2	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
0.3	16219617	15000	1081.3	1056	1023	1760	81.3	fc_backward_bias_kernel(const float *, float *, int, int)
0.3	14635034	15000	975.7	992	895	1504	85.4	zero_kernel(float *, int)
0.3	13688942	7500	1847.8	1824	1760	2816	140.5	softmax_cross_entropy_kernel(const float *, const float *, float *, float *, int, int)
0.1	7227938	7500	963.7	960	927	2080	71.3	scale_inplace_kernel(float *, int, float)

Figure 14: $Batch_S : 8, Block_S : 256$

1. GPU Active Time: 5077420 us
2. GPU Util : 62.5%

6 Optimization Techniques

6.1 Optimized Configuration

Each optimization is applied cumulatively:

1. **Optimization 1:** Shared memory tiling for convolution
2. **Optimization 2:** Kernel fusion (Conv+Bias+ReLU)
3. **Optimization 3:** Optimized backward pass (atomic reduction)
4. **Optimization 4:** Stream-based data transfer overlapping

6.2 Optimization 1: Shared Memory Tiling for Convolution

6.2.1 Motivation and Problem

The naive convolution implementation suffers from redundant global memory reads. Consider computing a 16×16 tile of output with a 3×3 kernel and stride=1:

- Each output pixel reads a 3×3 input window (9 values)
- $16 \times 16 = 256$ output pixels
- Total reads: $256 \times 9 = 2,304$ values
- But the input tile is only $18 \times 18 = 324$ unique values!

This redundancy wastes memory bandwidth and causes the kernel to be memory-bound rather than compute-bound.

6.2.2 Solution: Cooperative Loading into Shared Memory

The optimization uses shared memory as a cache:

```
1 // Each thread loads multiple elements using linear indexing
2 // tx, ty = threadIdx.x, threadIdx.y (0 to TILE_W-1, 0 to TILE_H
  -1)
3 int total_threads = TILE_H * TILE_W; // 16 * 16 = 256 threads
4 int total_elements = shH * shW; // 18 * 18 = 324 elements
5
6 // Linear thread ID within block
7 int tid = ty * TILE_W + tx;
8
9 // Each thread loads multiple elements in strided pattern
10 for (int s = tid; s < total_elements; s += total_threads) {
11     // Convert linear index to 2D coordinates in shared memory
12     int sh_y = s / shW; // Row in shared memory
13     int sh_x = s % shW; // Column in shared memory
14
15     // Map shared memory coordinates to global input coordinates
16     // oh0, ow0 = top-left output position for this tile
17     int ih = (oh0 * stride - pad) + sh_y;
18     int iw = (ow0 * stride - pad) + sh_x;
19
20     // Load from global memory with boundary checking (zero
      padding)
21     float v = 0.0f;
22     if (ih >= 0 && ih < input.H && iw >= 0 && iw < input.W) {
23         // NCHW indexing: ((n*C + ic)*H + ih)*W + iw
24         v = input.data[idx4(n, ic, ih, iw, input.C, input.H,
          input.W)];
25     }
26     // else v stays 0.0f (implicit zero padding)
```

```

27
28     // Store in shared memory (linear indexing for simplicity)
29     shmem[sh_y * shW + sh_x] = v;
30 }
31
32 // CRITICAL: Wait for all threads to finish loading
33 __syncthreads();

```

Listing 6: Cooperative Loading Pattern - Based on Our Implementation

Key Points of Cooperative Loading:

1. **Linear Indexing:** Thread with ID `tid` starts at element `tid`
2. **Strided Access:** Each thread loads elements at `tid`, `tid+256`, `tid+512`, etc.
3. **Coverage:** With 256 threads and 324 elements, each thread loads 1-2 elements
4. **Coalesced Access:** Adjacent threads load adjacent memory locations (fast!)
5. **Boundary Handling:** Out-of-bounds reads return 0.0f (padding)

Example: How 256 threads load 324 elements

```

1 Thread 0:   loads elements 0, 256
2 Thread 1:   loads elements 1, 257
3 Thread 2:   loads elements 2, 258
4 ...
5 Thread 67:  loads elements 67, 323
6 Thread 68:  loads element  68
7 ...
8 Thread 255: loads element  255
9
10 Total: 68 threads load 2 elements, 188 threads load 1 element
11       = 68*2 + 188*1 = 136 + 188 = 324 elements

```

Step 3: Compute convolution using fast shared memory

```

1 // Each thread computes one output pixel (if within bounds)
2 if (oh < outH && ow < outW) {
3     // Calculate starting position in shared memory for this
4     // output
5     // (oh - oh0) = position within tile (0 to TILE_H-1)
6     int sh_oh = (oh - oh0) * stride;
7     int sh_ow = (ow - ow0) * stride;
8
9     // Accumulate convolution result
10    float sum = 0.0f;
11    for (int kh = 0; kh < K; ++kh) {
12        for (int kw = 0; kw < K; ++kw) {
13            // Read from FAST shared memory (not global memory!)
14            float x = shmem[(sh_oh + kh) * shW + (sh_ow + kw)];
15
16            // Read weight from global memory

```

```

16         float w = weights.data[widx(oc, ic, kh, kw, input.C,
17                                   K)];
18
19         // Multiply-accumulate
20         sum += x * w;
21     }
22     acc += sum; // Accumulate across input channels
23 }
24
25 // Synchronize before loading next input channel
26 __syncthreads();

```

Listing 7: Convolution from Shared Memory

6.2.3 Why This Works

Memory Traffic Reduction:

- **Before:** Each of 256 threads reads 9 values = 2,304 global reads
- **After:** Cooperative load of 324 values + 256×9 shared reads
- **Global memory reduction:** $2,304 \rightarrow 324$ **7 $\times$ reduction**
- **Shared memory:** 100 $\times$ faster than global memory

Coalesced Memory Access:

The cooperative loading pattern creates coalesced access (adjacent threads load adjacent memory), maximizing memory bandwidth utilization.

6.2.4 Implementation Details

Key Parameters:

- **TILE_SIZE:** 16×16 (output tile size)
- **Shared memory size:** $(16+2) \times (16+2) = 324$ floats = 1.3 KB
- **Threads per block:** $16 \times 16 = 256$ threads
- **Halo region:** Extra border for kernel overlap

Critical Synchronization:

```

1 __syncthreads(); // After loading, before computing

```

This ensures all threads have loaded their data before any thread starts computing.

6.2.5 Expected Performance Impact

Actual Speedup:

- Kernel is not 100% memory-bound (some compute time)
- Shared memory bank conflicts reduce efficiency
- Synchronization overhead
- Other memory operations (weights, outputs) not optimized

6.3 Profiling:

In this section we will profile different configuration for block size and batch size that may effect our performance :

```
nsys profile --stats=true ./cnn --train-images train-images.idx3-ubyte
--train-labels train-labels.idx1-ubyte --test-images t10k-images.idx3-ubyte
--test-labels t10k-labels.idx1-ubyte --epochs 1 --batch 64 --lr 0.01 --block 256
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
65.2	272941019	1874	1456745.5	1808010.5	623673	4454172	833360.1	conv_dW_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int, int)
26.9	1126433197	1874	601085	798294	369531	1686028	237882.6	conv_db_kernel(const float *, float *, int, int, int, int, int, int, int, int, int, int, int, int)
2.9	120428301	1874	64262.7	95422.5	29472	544953	37533.4	conv_dX_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int, int)
1.7	71627676	2186	32766.5	36799	17375	72960	15350.8	conv_forward_tiled_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.6	65298473	2186	29871.2	31391	7008	79935	22762.4	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
0.3	13618364	1874	7267	7743.5	2528	17792	4680.9	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.3	13370358	1874	7134.7	7327.5	1503	19968	5634.6	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
0.2	8196480	7496	1093.4	960	895	5920	404.5	sgd_update_kernel(float *, float *, int, float, int)
0.2	8085293	2811	2876.3	2688	1056	12576	1687.9	relu_backward_kernel(const float *, const float *, float *, int)
0.2	7571908	3279	2309.2	2304	959	11200	1113.2	relu_forward_kernel(const float *, float *, int)
0.2	6342323	2186	2901.3	3392	2239	5312	642.3	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int, int, int, int)
0.1	5298289	1874	2827.3	3040	2272	5600	491.1	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int, int)
0.1	4229004	1874	2256.7	2656	1727	4000	496.8	zero_kernel(float *, int)
0.1	3334809	1874	1779.5	1824	1631	3936	185.4	fc_backward_bias_kernel(const float *, float *, int, int)
0	1832773	937	1956	1920	1856	2944	172.2	softmax_ce_loss_grad_kernel(const float *, const float *, float *, float *, int, int)
0	929723	937	992.2	992	959	1504	77.8	scale_offset_kernel(float *, int, float)

Figure 15: Batch_S : 64, Block_S : 256

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
82.3	4889632161	2811	1454867.4	7844.0	2140	6956017	2066277.7	cudaDeviceSynchronize
6.7	334160112	40	8354002.8	1860.0	1060	333396147	52711476.8	cudaMalloc
5.0	247767813	4068	60906.5	23703.5	9171	763400	68510.2	cudaMemcpy
4.9	245749428	37010	6640.1	3845.0	2523	7050209	37886.4	cudaLaunchKernel
0.9	45586650	8433	5405.7	4820.0	2330	356474	6765.8	cudaMemset
0.1	2770947	37010	74.9	58.0	48	32342	297.8	cuKernelGetName
0.0	1652498	40	41312.4	1674.5	779	447286	95975.5	cudaFree
0.0	505841	18	50584.1	54764.0	24344	78037	18867.9	cuLibraryLoadData
0.0	232906	10	23290.6	11304.0	6287	121080	34853.6	cuLibraryUnload
0.0	11140	1	11140.0	11140.0	11140	11140	0.0	cuModuleGetLoadingMode
0.0	3600	16	225.0	223.5	123	328	60.0	cuLibraryGetKernel
0.0	2954	1	2954.0	2954.0	2954	2954	0.0	cuCtxSynchronize
0.0	834	1	834.0	834.0	834	834	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\Quad\Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\pr
ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
73.0	18886743	2038	9267.3	16704.0	287	32608	8306.1	[CUDA memcpy Host-to-Device]
20.1	5191679	8433	615.6	352.0	287	5024	342.6	[CUDA memset]
6.9	1784681	2030	879.2	864.0	767	1824	85.4	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
386.115	8433	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.021	2038	0.108	0.201	0.000	0.401	0.100	[CUDA memcpy Host-to-Device]
2.802	2030	0.001	0.003	0.000	0.003	0.001	[CUDA memcpy Device-to-Host]

Figure 16: Batch_S : 64, Block_S : 256

1. GPU Active Time: 4181863 us

2. GPU Util : 93.6%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
  512
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
50.1	2738167802	15000	182544.5	198959.5	84159	1128179	103604	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
20.4	1113805399	15000	74253.7	85168	47009	499531	27588.2	conv_db_kernel(const float *, float *, int, int, int, int)
16.1	879833855	17500	50276.2	50863.5	3615	149731	46880.2	forward_kernel(const float *, const float *, const float *, float *, int, int, int)
5.9	321883515	15000	21458.9	27935	14271	494575	9142.8	conv_dx_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
2.2	118591912	17500	6776.7	8096	3647	14720	2979.5	conv_forward_tiled_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.2	64208112	60000	1070.1	960	927	7584	280.3	sgd_update_kernel(float *, float *, int, float, int)
0.9	46706407	15000	3113.8	3663.5	1471	9568	1605.5	backward_input_kernel(const float *, const float *, float *, int, int, int)
0.6	33368383	15000	2224.6	2544	1311	4768	900.5	backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.5	27991705	17500	1599.5	1600	1535	3008	80.5	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.5	27315131	26250	1040.6	1024	959	3040	81.7	relu_forward_kernel(const float *, float *, int)
0.5	26477767	22500	1176.8	1152	1056	5312	103.1	relu_backward_kernel(const float *, const float *, float *, int)
0.4	20948791	15000	1396.6	1376	1312	3456	82.3	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
0.3	15917395	15000	1061.2	1056	1023	1632	58.9	backward_bias_kernel(const float *, float *, int, int)
0.3	13610007	7500	1841.3	1824	1760	33200	522.2	softmax_cross_entropy_kernel(const float *, const float *, float *, float *, int, int)
0.3	13685661	15000	912.4	928	831	3264	58.9	zero_kernel(float *, int)
0.1	7170191	7500	956	960	896	2304	53.2	scale_inplace_kernel(float *, int, float)

Figure 17: Batch_s : 8, Block_s : 512

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
47.8	4061401690	22500	180506.7	8540.0	2163	1584384	245412.8	cudaDeviceSynchronize
23.6	2004527177	296250	6766.3	3860.0	2457	2060369	18331.6	cudaLaunchKernel
18.5	1573397995	32508	48400.3	12620.0	4223	4257307	63114.8	cudaMemcpy
5.1	435785698	40	10894642.4	2712.5	1827	435439482	68847629.1	cudaMalloc
4.7	399950820	67500	5925.2	3930.0	2209	623755	7463.7	cudaMemset
0.3	23821526	296250	77.7	58.0	48	164898	507.6	cuKernelGetName
0.0	800029	10	80002.0	73978.0	58305	122068	25439.4	cuLibraryLoadData
0.0	448348	40	11086.5	1335.5	769	262828	44803.7	cudaFree
0.0	217889	10	21788.0	10586.5	6547	113409	32774.1	cuLibraryUnload
0.0	8959	1	8959.0	8959.0	8959	8959	0.0	cuModuleGetLoadingMode
0.0	6098	16	381.1	344.5	271	615	106.4	cuLibraryGetKernel
0.0	2617	1	2617.0	2617.0	2617	2617	0.0	cuCtxSynchronize
0.0	299	1	299.0	299.0	299	299	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
 [libprotobuf ERROR C:\vs\p4\build\sw\devtools\Agona\Rel\Quad0_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\protobuf\wire_data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.]
 [6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
51.7	41753093	67500	618.6	394.0	287	13888	329.8	[CUDA memset]
31.1	25141352	16258	1546.4	2368.0	287	32705	1165.1	[CUDA memcpy Host-to-Device]
17.2	13922966	16258	856.8	832.0	736	15775	133.0	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.579	67500	0.846	0.800	0.800	0.481	0.126	[CUDA memset]
220.172	16258	0.814	0.825	0.800	0.481	0.013	[CUDA memcpy Host-to-Device]
2.830	16258	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 18: Batch_s : 8, Block_s : 512

1. GPU Active Time: 5462704 us

2. GPU Util : 67.9%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
  1024
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
43.8	2996113645	15000	193074.2	221309	110655	531866	81363.3	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
25.9	1712223807	17500	97841.4	97406.5	3584	284396	94549.9	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
16.9	1116182637	15000	74412.8	85125	47007	149407	27437.4	conv_db_kernel(const float *, float *, int, int, int, int)
6.2	410822654	15000	27388.2	32015.5	18463	54880	8848.5	conv_dx_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
1.8	118608676	17500	6777.5	7680	3648	14720	2995.9	conv_forward_tiled_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.1	72013732	15000	4800.9	5183.5	1599	12160	3172.5	backward_input_kernel(const float *, const float *, float *, int, int, int)
1.	69311553	60000	1155.2	992	927	3296	412.4	sgd_update_kernel(float *, float *, int, float, int)
0.6	41455140	15000	2763.7	3135.5	1535	5790	1195.7	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.5	33343493	17500	1904.9	1888	1823	2880	114.3	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.5	30588055	26250	1164.1	1024	991	2176	204.4	relu_forward_kernel(const float *, float *, int)
0.5	29845245	22500	1326.5	1152	1119	2464	250.9	relu_backward_kernel(const float *, const float *, float *, int)
0.4	2469204	15000	1646	1832	1568	2464	105.5	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
0.2	16167425	15000	1077.8	1056	1023	1664	71.5	backward_bias_kernel(const float *, float *, int, int)
0.2	15362365	15000	1024.2	1120	832	1728	149.4	zero_kernel(float *, int)
0.2	13900777	7500	1853.4	1824	1760	2848	120.8	softmax_ce_loss_grad_kernel(const float *, const int *, float *, float *, int, int)
0.1	7443404	7500	992.5	992	959	1472	62.7	scale_inplace_kernel(float *, int, float)

Figure 19: Batch_S : 8, Block_S : 1024

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
45.1	4319223598	22500	191965.5	7667.0	2285	1067750	262528.0	cudaDeviceSynchronize
24.9	2383179948	32588	73310.6	11225.0	4327	1192249	182819.2	cudaMemcpy
18.0	1726848984	296250	5829.0	3659.0	2448	85899	9810.0	cudaLaunchKernel
8.8	843404923	40	21085123.1	1357.0	915	843265966	133331493.3	cudaMalloc
2.9	280938326	67500	4161.9	3659.0	2360	324958	3700.3	cudaMemset
0.2	18714429	296250	63.2	55.0	47	47826	246.5	cudaKernelGetName
0.0	540087	40	13502.2	1391.0	821	314577	54455.0	cudaFree
0.0	308767	10	30876.7	29628.0	18862	48034	8668.3	cuLibraryLoadData
0.0	223796	10	22379.6	9867.0	6659	113343	32534.4	cuLibraryUnload
0.0	8521	1	8521.0	8521.0	8521	8521	0.0	cuModuleGetLoadingMode
0.0	3699	16	225.6	212.5	191	323	68.3	cuLibraryGetKernel
0.0	2528	1	2528.0	2528.0	2528	2528	0.0	cuCtxSynchronize
0.0	349	1	349.0	349.0	349	349	0.0	cuDeviceGetLuid

```

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3.21.1\src\google\protobuf\
data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

```

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
51.9	42324733	67500	627.0	384.0	287	14048	333.8	[CUDA memset]
31.0	25338688	16258	1558.5	2336.0	287	32351	1239.4	[CUDA memcpy Host-to-Device]
17.1	13951011	16250	858.5	864.0	767	14432	184.9	[CUDA memcpy Device-to-Host]

```

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

```

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.578	67500	0.045	0.000	0.000	0.401	0.126	[CUDA memset]
220.172	16258	0.014	0.025	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
2.830	16258	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 20: Batch_S : 8, Block_S : 1024

1. GPU Active Time: 6524291 us
2. GPU Util : 83.2%

```

1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
  256

```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
54.3	2736286471	15000	182419.1	215309	78431	1182706	112813.8	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
22.1	1116830061	15000	74455.3	87390.5	46911	498266	28058.7	conv_db_kernel(const float *, float *, int, int, int, int)
9.6	483445072	17500	27625.4	28808	3615	79711	23969.1	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
5.9	296457044	15000	19763.9	26047	12863	248765	7103.8	conv_dx_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
2.4	119177496	17500	6810.1	7616	3648	16900	2991.2	conv_forward_tiled_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.3	63888774	60000	1066.5	960	895	5600	296.4	sgd_update_kernel(float *, float *, int, float, int)
0.8	42137022	15000	2809.1	4064	1376	15424	1389.1	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
0.6	32055213	15000	2137	2912	1183	5280	943.1	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.5	27544498	26250	1049.3	1056	895	7072	127.3	relu_forward_kernel(const float *, float *, int)
0.5	26908803	17500	1537.6	1530	1439	6048	118.7	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.5	26455408	22500	1175.8	1152	1023	5952	138.9	relu_backward_kernel(const float *, const float *, float *, int)
0.4	20070982	15000	1338.1	1344	1215	18569	237.8	maxpool_backward_kernel(const float *, const int *, float *, int, int, int, int, int, int, int, int)
0.3	16085671	15000	1072.4	1056	1023	4544	83.7	fc_backward_bias_kernel(const float *, float *, int, int)
0.3	14597029	15000	973.2	992	895	5728	92.5	zero_kernel(float *, int)
0.3	13847935	7500	1846.4	1824	1760	2816	127.1	softmax_ce_loss_grad_kernel(const float *, const int *, float *, float *, int, int)
0.1	7264680	7500	968.6	960	927	2784	67.7	scale_inplace_kernel(float *, int, float)

Figure 21: Batch_S : 8, Block_S : 256

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
50.0	4019027976	22500	178623.5	7272.0	1984	1441717	245800.1	cudaDeviceSynchronize
23.2	1865126641	296250	6295.8	3791.0	2466	1094103	8084.7	cudaLaunchKernel
14.1	1130865798	32508	34787.3	14037.0	4186	3760552	46157.5	cudaMemcpy
8.1	648909233	40	16222730.8	1501.0	942	648742988	102574599.6	cudaMalloc
4.4	352540646	67500	5222.8	3790.0	2171	219054	5175.9	cudaMemset
0.3	21567925	296250	72.8	57.0	47	95529	356.0	cuKernelGetName
0.0	679931	40	16998.3	1309.0	818	379469	68905.7	cudaFree
0.0	460174	10	46017.4	24186.5	16830	194519	55478.8	cuLibraryUnload
0.0	350339	10	35033.9	29741.0	21128	67639	13913.1	cuLibraryLoadData
0.0	9068	1	9068.0	9068.0	9068	9068	0.0	cuModuleGetLoadingMode
0.0	3412	16	213.3	206.0	129	299	56.1	cuLibraryGetKernel
0.0	3262	1	3262.0	3262.0	3262	3262	0.0	cuCtxSynchronize
0.0	363	1	363.0	363.0	363	363	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agona\Rel\Quad_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\p
ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
52.1	41548705	67500	615.5	384.0	256	3968	334.4	[CUDA memset]
30.7	24495052	16258	1506.6	2272.0	287	32480	1140.1	[CUDA memcpy Host-to-Device]
17.2	13713212	16250	843.9	832.0	736	1824	54.4	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.172	16258	0.014	0.025	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
2.830	16250	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 22: $Batch_S : 8, Block_S : 256$

1. GPU Active Time: 5009579 us
2. GPU Util : 62.3%

6.4 Optimization 2: Kernel Fusion (Conv + Bias + ReLU)

6.4.1 Problem: Intermediate Memory Operations

The naive implementation uses separate kernels:

```

1 // Two separate kernel launches
2 conv_forward_naive_kernel<<<blocksPerGrid, threadsPerBlock, 0, s
  >>>(*d_input, *d_weights, d_bias, *d_output, pad, stride,
    total_output_elements);
3 relu_forward_kernel<<<blocks, threads, 0, s>>>(X.data, Y.data,
  N);

```

Listing 8: Naive Separate Kernels

This causes:

- 4 global memory operations:
 1. Read input (conv)
 2. Write conv_output
 3. Read conv_output (bias)
 4. Write final_output
- 2 kernel launch overheads
- No data reuse: Intermediate results written then immediately read

6.4.2 Solution: Fused Kernel

Combine all three operations in a single kernel:

```
1 __global__ void conv_forward_tiled_fused_kernel(  
2     Tensor input,  
3     Tensor weights,  
4     const float* __restrict__ bias,  
5     Tensor output,  
6     int K, int pad, int stride  
7 ){  
8     {  
9         // Step 1 :Convolutio  
10  
11        // Step 2: Add bias (no memory write!)  
12        sum += bias[channel];  
13  
14        // Step 3: Apply ReLU (no memory write!)  
15        float result = (sum > 0.0f) ? sum : 0.0f;  
16  
17        // Single write to global memory  
18        output[idx] = result;  
19    }  
20 }
```

Listing 9: Fused Conv+Bias+ReLU Kernel

6.4.3 Benefits

Memory Traffic Reduction:

- **Before:** 2 writes + 2 reads = 4 intermediate memory ops
- **After:** 1 final write = 1 memory op
- **Reduction:** 75% less memory traffic

Intermediate values stay in fast registers instead of being written to slow global memory.

6.4.4 Expected Performance Impact

- Convolution still dominates (compute + memory for input/weights)
- Bias and ReLU are cheap operations
- But eliminating memory traffic is always beneficial

6.5 Profiling:

In this section we will profile different configuration for block size and batch size that may effect our performance :

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 64 --lr 0.01 --
  block 256
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
65.3	2809024459	1874	1498945.8	1780139	621945	5330658	872072.3	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
27.1	1163850906	1874	621051.7	798230	369564	3180123	272334.5	conv_dw_kernel(const float *, float *, int, int, int, int)
2.8	122506461	1874	65371.6	95423	29471	426715	38689.4	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
1.7	72207910	2186	33032	36879	17375	72575	15565.1	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.5	65679412	2186	30045.5	31360	6976	79807	22982.4	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
0.3	13735662	1874	7329.6	8672	2527	17951	4767.5	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.3	13491403	1874	7199.3	8288	1503	19711	5695.3	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
0.2	8134992	7496	1085.1	960	885	5759	375.8	spt_update_kernel(float *, float *, int, float, int)
0.2	7160871	2811	2547.4	2272	1055	11935	1545.6	relu_backward_kernel(const float *, const float *, float *, int)
0.1	6415111	2186	2934.6	3360	10112	10112	660.7	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.1	5287334	1874	2821.4	3040	2240	6432	516.7	maxpool_backward_kernel(const float *, const float *, const float *, int, int, int, int, int, int, int, int)
0.1	4245714	1874	2265.6	2556	1696	3936	510.1	zero_kernel(float *, int)
0.1	3355125	1874	1790.4	1824	1631	3584	207.2	fc_backward_bias_kernel(const float *, float *, int, int)
0	1862134	937	1987.3	1952	1824	4576	222.8	softmax_loss_grad_kernel(const float *, const float *, float *, float *, int, int)
0	1080537	1093	988.6	960	927	1472	93.1	relu_forward_kernel(const float *, float *, int)
0	923058	937	985.1	960	928	1440	93.3	scale_inplace_kernel(float *, int, float)

Figure 23: Batch_S : 64, Block_S : 256

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
81.3	4180581255	2811	1487193.6	9107.0	2292	9872836	2114756.9	cudaDeviceSynchronize
6.2	317788481	34824	9125.6	4195.0	2429	6950956	39609.4	cudaLaunchKernel
6.0	307866749	4068	75680.1	33051.5	8023	649124	84852.9	cudaMemcpy
5.0	256366839	40	6409151.0	1638.5	923	255544610	40401850.9	cudaMalloc
1.5	75664183	8433	8972.4	4583.0	2121	479960	12064.5	cudaMemset
0.1	4052501	34824	116.4	63.0	48	165804	1136.9	cuKernelGetName
0.0	2250766	40	56269.2	1607.5	816	420906	120621.4	cudaFree
0.0	394428	10	39442.8	37713.0	22540	58523	12907.2	cuLibraryLoadData
0.0	224069	10	22406.9	9554.5	7412	111916	32065.9	cuLibraryUnload
0.0	13895	1	13895.0	13895.0	13895	13895	0.0	cuModuleGetLoadingMode
0.0	4819	16	251.2	252.5	119	457	93.5	cuLibraryGetKernel
0.0	2729	1	2729.0	2729.0	2729	2729	0.0	cuCtxSynchronize
0.0	456	1	456.0	456.0	456	456	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
 [libprotobuf ERROR C:\dev\p4\build\sw\devtools\Agora\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\pr
 ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
 [6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
73.1	19046996	2038	9345.9	16704.0	288	32544	8377.1	[CUDA memcpy Host-to-Device]
19.6	5117632	8433	606.9	384.0	287	3776	344.5	[CUDA memset]
7.3	1894126	2030	933.1	896.0	768	14240	429.3	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
386.115	8433	0.046	0.009	0.000	0.401	0.126	[CUDA memset]
220.021	2038	0.108	0.201	0.000	0.401	0.100	[CUDA memcpy Host-to-Device]
2.802	2030	0.001	0.003	0.000	0.003	0.001	[CUDA memcpy Device-to-Host]

Figure 24: Batch_S : 64, Block_S : 256

1. GPU Active Time: 4296950 us

2. GPU Util : 83.51%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
  512
```


Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
49.9	2722364324	15000	181491	198733	84287	886486	100410.7	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
20.6	1121710695	15000	74781.3	85039.5	46911	497914	29297.7	conv_db_kernel(const float *, float *, int, int, int, int)
16.2	884658788	17500	50563.4	50815.5	3615	148830	47092.9	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
5.9	324049231	15000	21603.3	27936	14272	375708	9161.1	conv_dx_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
2.2	119819470	17500	6846.8	7792	3648	14784	3033.6	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.2	64618705	60000	1077	960	896	6368	295.1	sgd_update_kernel(float *, float *, int, float, int)
0.9	46881906	15000	3125.4	3603.5	1471	11172	1624.1	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
0.6	33276102	15000	2218.4	2543.5	1247	5056	937.1	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.5	27983089	17500	1599	1568	1536	2464	112.7	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.5	24655728	22500	1095.8	1088	991	3072	107.2	relu_backward_kernel(const float *, const float *, float *, int)
0.4	20832794	15000	1388.9	1376	1311	2560	103.6	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
0.3	16030810	15000	1068.9	1056	1023	1664	79.9	fc_backward_bias_kernel(const float *, float *, int, int)
0.3	13864326	7500	1848.6	1824	1728	2816	138.7	softmax_loss_grad_kernel(const float *, const int *, float *, float *, int, int)
0.3	13734406	15000	915.6	928	832	1440	71.7	zero_kernel(float *, int)
0.2	8531996	8750	975.1	960	928	2240	66.3	relu_forward_kernel(const float *, float *, int)
0.1	7255364	7500	967.4	960	927	1920	68.8	scale_inplace_kernel(float *, int, float)

Figure 25: $Batch_S : 8, Block_S : 512$

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
50.7	4054455873	22500	180198.0	7797.0	2175	1184095	246230.4	cudaDeviceSynchronize
22.5	1804180257	278750	6472.1	3711.0	2527	1493623	10433.8	cudaLaunchKernel
20.3	1625293139	32500	49996.9	11355.0	3699	1315793	64346.1	cudaMemcpy
4.1	338941115	67500	4982.8	3697.0	2271	242590	5413.3	cudaMemset
2.1	165712663	40	4143066.6	1243.0	934	165534689	26175817.4	cudaMalloc
0.3	28286404	278750	72.8	58.0	48	62897	375.3	cuKernelGetName
0.0	651168	40	16279.2	1338.5	833	384521	67398.7	cudaFree
0.0	350621	10	35062.1	32376.5	18978	54764	11636.1	cuLibraryLoadData
0.0	253205	10	25320.5	11980.5	8871	121120	34621.3	cuLibraryUnload
0.0	8576	1	8576.0	8576.0	8576	8576	0.0	cuModuleGetLoadingMode
0.0	3476	16	217.3	220.0	100	361	73.7	cuLibraryGetKernel
0.0	3072	1	3072.0	3072.0	3072	3072	0.0	cuCtxSynchronize
0.0	330	1	330.0	330.0	330	330	0.0	cuDeviceGetLuid

```

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\Quad_Main\Imports\Source\ProtoBuf\protobuf-3.21.1\src\google\proto
ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

```

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
52.5	42362194	67500	627.6	448.0	287	3168	328.3	[CUDA memset]
30.4	24568781	16258	1511.2	2384.0	287	32447	1152.1	[CUDA memcpy Host-to-Device]
17.1	13803281	16250	849.4	832.0	767	1664	46.9	[CUDA memcpy Device-to-Host]

```

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

```

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.172	16258	0.014	0.025	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
2.830	16250	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 26: $Batch_S : 8, Block_S : 512$

1. GPU Active Time: 5734764 us
2. GPU Util : 71.7%

```

nsys profile --stats=true ./cnn --train-images train-images.idx3-
ubyte --train-labels train-labels.idx1-ubyte --
test-images t10k-images.idx3-ubyte --test-labels t10k-
labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
1024

```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
44.2	2918536671	15000	194569.1	221165	110655	808214	86067.9	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
25.9	1710335908	17500	97733.5	97487	3583	294909	94396.1	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
16.9	1113232876	15000	74215.5	85071	46975	149246	27256.3	conv_db_kernel(const float *, float *, int, int, int, int)
6.2	409893847	15000	27326.3	31999.5	18463	55072	8782.8	conv_dx_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
1.8	117820707	17500	6722.6	7599.5	3648	14688	2946.0	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.1	71786425	15000	4785.8	6095.5	1599	12160	3153.1	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
1	68865847	60000	1147.8	960	927	7008	416.3	sgd_update_kernel(float *, float *, int, float, int)
0.6	41389215	15000	2759.3	3135.5	1535	10080	1188.6	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.5	33274675	17500	1901.4	1888	1824	4416	111.8	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.4	27542166	22500	1224.1	1152	902	3808	194.1	relu_backward_kernel(const float *, const float *, float *, int)
0.4	24551444	15000	1636.8	1632	1568	2624	96.1	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
0.2	16125748	15000	1075	1056	1023	2431	67.1	fc_backward_bias_kernel(const float *, float *, int, int)
0.2	15310740	15000	1020.7	1120	832	2176	147.8	zero_kernel(float *, int)
0.2	13744332	7500	1832.6	1824	1760	10080	147.8	softmax_loss_grad_kernel(const float *, const int *, float *, float *, int, int)
0.1	8910634	8750	1018.4	1024	991	1696	55.2	relu_forward_kernel(const float *, float *, int)
0.1	7372632	7500	983	992	928	2080	58.2	scale_inplace_kernel(float *, int, float)

Figure 27: $Batch_S : 8, Block_S : 1024$

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
42.7	4166590562	22500	185181.8	8357.0	2018	2200137	253948.7	cudaDeviceSynchronize
24.3	2372292314	32508	72975.6	17877.5	4194	4473927	99653.9	cudaMemcpy
21.9	2141177891	278750	7681.4	4280.0	2432	837716	9387.2	cudaLaunchKernel
6.4	621265319	40	15531545.5	1103.5	916	621608930	9820267.0	cudaMalloc
4.5	436459034	67500	6466.1	3972.0	2165	301194	7256.0	cudaMemset
0.3	24794598	278750	88.9	61.0	48	122953	404.8	cuKernelGetName
0.0	889223	40	22230.6	1296.5	811	537361	94774.3	cudaFree
0.0	345342	10	34534.2	31603.0	21139	50473	10683.9	cuLibraryLoadData
0.0	212337	10	21233.7	10961.5	6140	103636	29604.0	cuLibraryUnload
0.0	56821	1	56821.0	56821.0	56821	56821	0.0	cuModuleGetLoadingMode
0.0	4496	16	281.0	210.5	131	1028	211.1	cuLibraryGetKernel
0.0	2088	1	2088.0	2088.0	2088	2088	0.0	cuCtxSynchronize
0.0	451	1	451.0	451.0	451	451	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dev\p4\build\sw\devtools\Agona\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\protobuf\src\api.cc:124: ParseAndMergeFrom: Cannot parse from the stream. Invalid data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
52.5	41707576	67500	617.9	384.0	287	4128	329.7	[CUDA memset]
30.1	23923996	16258	1471.5	2240.0	287	32352	1165.0	[CUDA memcpy Host-to-Device]
17.4	13861252	16250	853.0	832.0	736	14336	166.4	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.172	16258	0.014	0.025	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
2.830	16250	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 28: $Batch_S : 8, Block_S : 1024$

1. GPU Active Time: 6558107 us
2. GPU Util : 67.17%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-ubyte
  --train-labels train-labels.idx1-ubyte --
test-images t10k-images.idx3-ubyte --test-labels t10k-labels.idx1-ubyte
  --epochs 1 --batch 8 --lr 0.01 --block 256
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
54.2	2705962201	15000	180337.5	208813	78559	1305664	108813.6	conv_dW_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
22.4	1117746679	15000	74516.4	85102.5	46975	504026	28372.4	conv_db_kernel(const float *, float *, int, int, int, int)
9.7	483710132	17500	27640.6	28079.5	3584	79392	23961.9	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
5.9	296808706	15000	19787.2	26047	12863	369788	7866.4	conv_dX_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
2.4	119529977	17500	6830.3	9295.5	3679	14784	2990.9	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
1.3	64289933	60000	1071.5	960	895	7264	318.2	sgd_update_kernel(float *, float *, int, float, int)
0.8	42326044	15000	2821.7	4095	1407	13695	1398.4	backward_input_kernel(const float *, const float *, float *, int, int, int)
0.6	32360095	15000	2157.3	2927.5	1183	12416	958.9	backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.5	26796512	17500	1531.2	1536	1439	10176	122.6	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.5	25047437	22500	1113.2	1056	1023	5952	127.5	relu_backward_kernel(const float *, const float *, float *, int)
0.4	20202491	15000	1346.8	1376	1215	24000	299.8	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
0.3	16217667	15000	1081.2	1056	1023	3520	69.9	backward_bias_kernel(const float *, float *, int, int)
0.3	14670294	15000	978	992	895	4768	85.1	zero_kernel(float *, int)
0.3	13888296	7500	1851.8	1824	1728	18496	228.2	softmax_loss_grad_kernel(const float *, const float *, float *, float *, int, int)
0.2	8240295	8750	941.7	928	895	1760	52.5	relu_forward_kernel(const float *, float *, int)
0.1	7381970	7500	984.3	960	959	3488	65.3	scale_grad_kernel(float *, int, float)

Figure 29: $Batch_S : 8, Block_S : 256$

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
54.4	3988869494	22500	177283.1	8461.0	2209	1638699	242821.9	cudaDeviceSynchronize
23.6	1728681119	278750	6201.5	3702.0	2480	3598645	10056.5	cudaLaunchKernel
15.5	1137687355	32508	34997.2	13838.5	4225	878885	39528.2	cudaMemcpy
4.8	352320318	67500	5219.6	3648.0	2158	352339	5431.2	cudaMemset
1.3	96818249	40	2420456.2	1303.0	985	96674635	15285023.4	cudaMalloc
0.3	20845561	278750	74.8	58.0	48	218611	593.8	cuKernelGetName
0.0	437532	40	10930.3	1259.0	760	254618	43695.9	cudaFree
0.0	367525	10	36752.5	31095.5	21599	61149	14308.4	cuLibraryLoadData
0.0	262045	10	26204.5	11199.5	9102	132443	38170.5	cuLibraryUnload
0.0	9129	1	9129.0	9129.0	9129	9129	0.0	cuModuleGetLoadingMode
0.0	3304	16	206.5	183.5	114	349	69.6	cuLibraryGetKernel
0.0	2843	1	2843.0	2843.0	2843	2843	0.0	cuCtxSynchronize
0.0	393	1	393.0	393.0	393	393	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\Quad_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\pi
ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
52.4	43298812	67500	641.5	704.0	287	16031	338.3	[CUDA memset]
30.3	25810334	16258	1538.3	2304.0	256	32864	1163.6	[CUDA memcpy Host-to-Device]
17.3	14283523	16250	879.0	864.0	767	15616	144.4	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.172	16258	0.014	0.025	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
2.830	16250	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 30: $Batch_S : 8, Block_S : 256$

1. GPU Active Time: 4978762 us
2. GPU Util : 68%

```

1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 32 --lr 0.01 --
  block 256

```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
63.7	2857991737	3750	762051.1	1007923.5	312221	2787552	439838	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
26.2	1174595005	3750	313225.3	388779.5	105214	1056020	140677.9	conv_dw_kernel(const float *, float *, int, int, int, int)
3.3	147214048	3750	39257.3	55839	10908	510938	31064.1	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
2.9	12890225	4374	29421.6	31087.5	7007	79519	22423.6	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
1.8	78942616	4374	18048.2	21120	9472	40064	8465.3	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
0.4	19380015	3750	5108	5455.5	1439	18144	3695.5	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
0.4	16466455	15000	1097.8	960	895	6560	383.6	update_kernel(float *, float *, int, float, int)
0.4	15708755	3750	4189	5552	1759	12415	2379.1	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.2	9779723	5625	1738.6	1632	1024	6624	636.6	relu_backward_kernel(const float *, const float *, float *, int)
0.2	9004376	4374	2058.6	2272	1727	6976	333.3	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
0.2	7554250	3750	2014.5	2144	1000	46751	785.3	maxpool_backward_kernel(const float *, const int *, float *, int, int, int, int, int, int, int, int)
0.1	5820318	3750	1552.1	1728	1279	6272	289.5	zero_kernel(float *, int)
0.1	5074559	3750	1353.2	1344	1248	3808	117.6	fc_backward_bias_kernel(const float *, float *, int, int)
0.1	3677634	1875	1961.4	1920	1887	29472	654.2	softmax_cross_entropy_loss_kernel(const float *, const int *, float *, float *, int, int)
0	2107712	2187	963.7	960	927	1440	60.6	relu_forward_kernel(const float *, float *, int)
0	1854056	1875	988.8	992	959	1536	65.7	scale_inplace_kernel(float *, int, float)

Figure 31: $Batch_S : 32, Block_S : 256$

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
68.6	4198284654	5625	746361.7	8720.0	2356	3663626	1053834.3	cudaDeviceSynchronize
12.6	779016132	40	19250483.3	2769.5	1860	769577876	121679172.5	cudaMalloc
10.2	623533567	69684	8940.3	5990.0	2507	3643085	17244.5	cudaLaunchKernel
6.4	389368396	8132	47873.6	24656.0	5783	3482929	63265.1	cudaMemcpy
2.1	126479034	16875	7495.1	5036.0	2481	152861	7502.7	cudaMemset
0.1	6922657	69684	99.3	73.0	48	31184	322.5	cuKernelGetName
0.0	1315836	40	32895.9	1442.5	1035	472945	100479.4	cudaFree
0.0	578901	10	57890.1	54053.0	37318	81509	14358.6	cuLibraryLoadData
0.0	283117	10	28311.7	15476.5	8596	115980	33098.3	cuLibraryUnload
0.0	9117	1	9117.0	9117.0	9117	9117	0.0	cuModuleGetLoadingMode
0.0	5683	16	355.2	356.5	245	553	73.7	cuLibraryGetKernel
0.0	2944	1	2944.0	2944.0	2944	2944	0.0	cuCtxSynchronize
0.0	307	1	307.0	307.0	307	307	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dev\p4\build\sw\devtools\Agora\Rel\Quad_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\protobuf\src\api.cc:124: void protobuf::Message::ParseFromArray(const void*, int): Invalid data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
59.7	20601125	4070	5061.7	8960.0	287	32415	4406.0	[CUDA memcpy Host-to-Device]
30.0	10350717	16875	613.4	352.0	287	14016	371.5	[CUDA memcpy]
10.4	3578864	4062	881.1	864.0	767	25440	401.4	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
772.643	16875	0.046	0.000	0.000	0.401	0.126	[CUDA memcpy]
220.122	4070	0.054	0.100	0.000	0.401	0.050	[CUDA memcpy Host-to-Device]
2.007	4062	0.001	0.001	0.000	0.001	0.001	[CUDA memcpy Device-to-Host]

Figure 32: Batch_S : 32, Block_S : 256

1. GPU Active Time: 4475917 us
2. GPU Util : 73.1%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-ubyte
  --train-labels train-labels.idx1-ubyte --
test-images t10k-images.idx3-ubyte --test-labels t10k-labels.idx1-ubyte
  --epochs 1 --batch 64 --lr 0.01 --block 512
```

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
04.5	2735027854	1874	1459459.0	1830762.5	669652	5020140	709389.1	conv_dW_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
26.4	1119474611	1874	597357.3	798167	369659	1731019	233274.7	conv_db_kernel(const float *, float *, int, int, int, int)
2.9	122834408	1874	65546.6	71951.5	28448	738711	39836.3	conv_dX_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
2.9	120881348	2186	55298	58079	12767	149118	42688.3	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
1.7	71623546	2186	32764.7	36963.5	17408	72963	15208	conv_forward_tiled_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
0.3	14751029	1874	7871.4	9168	3071	19616	4744.8	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
0.3	13888219	1874	7411	8352	1503	19488	5886.7	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
0.2	8251696	7496	1100.8	960	927	6783	365.4	sgd_update_kernel(float *, float *, int, float, int)
0.2	7093832	2811	2523.6	2304	1056	12224	1485.5	relu_backward_kernel(const float *, const float *, float *, int)
0.2	6648911	2186	3041.6	3504	2303	8032	708.4	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int)
0.1	5240832	1874	2796.6	3136	2208	5664	532.8	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int)
0.1	4256686	1874	2271.4	2656	1728	4096	483.1	zero_kernel(float *, int)
0.1	3332124	1874	1778.1	1824	1631	2816	170.6	fc_backward_bias_kernel(const float *, float *, int, int)
0	1851212	937	1975.7	1920	1888	20000	623.9	softmax_loss_grad_kernel(const float *, const int *, float *, float *, int, int)
0	1071347	1093	980.2	960	959	1472	71.3	relu_forward_kernel(const float *, float *, int)
0	940856	937	1004.1	992	960	1504	73.8	scale_inplace_kernel(float *, int, float)

Figure 33: Batch_S : 64, Block_S : 512

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
70.9	4045997753	2811	1439344.6	8446.0	2328	7434754	2040045.4	cudaDeviceSynchronize
15.1	864757246	40	21618931.1	3919.5	2094	863633510	136547907.6	cudaMalloc
6.5	369071344	4068	90725.5	30286.0	9088	832993	109266.3	cudaMemcpy
6.2	353794649	34824	10159.5	7761.0	2388	7110819	39975.9	cudaLaunchKernel
1.2	70873768	8433	8404.3	6815.0	2502	228156	8561.2	cudaMemset
0.1	3666156	34824	105.3	105.0	47	80437	543.0	cuKernelGetName
0.0	1289345	40	32233.6	1612.0	891	304506	70265.7	cudaFree
0.0	682241	10	68224.1	65293.5	42901	97484	18212.2	cuLibraryLoadData
0.0	221123	10	22112.3	9635.0	6121	126105	36893.3	cuLibraryUnload
0.0	8845	1	8845.0	8845.0	8845	8845	0.0	cuModuleGetLoadingMode
0.0	6162	16	385.1	370.0	256	538	100.2	cuLibraryGetKernel
0.0	2171	1	2171.0	2171.0	2171	2171	0.0	cuCtxSynchronize
0.0	344	1	344.0	344.0	344	344	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\protobuf\src\util.cc:124: ParseError: Unexpected end of data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
73.7	18991387	2038	9318.6	16703.5	288	41632	8420.4	[CUDA memcpy Host-to-Device]
19.3	4982827	8433	590.9	352.0	287	3008	343.8	[CUDA memset]
7.0	1800136	2030	886.8	864.0	768	3296	131.0	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
386.115	8433	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.021	2038	0.108	0.201	0.000	0.401	0.100	[CUDA memcpy Host-to-Device]
2.802	2030	0.001	0.003	0.000	0.003	0.001	[CUDA memcpy Device-to-Host]

Figure 34: Batch_S : 64, Block_S : 512

1. GPU Active Time: 4235282 us
2. GPU Util : 75.1%

Tile size configuration : In all previous part Tile size was 16*16 , in this part we change it to 8*8:

Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
2691105791	1874	1436022.3	1577581.5	667608	3214235	767293.4	conv_dW_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
1115082990	1874	595028.3	707751.5	368891	1190738	225916.9	conv_db_kernel(const float *, float *, int, int, int)
122304553	1874	65584.1	71998.5	28449	154750	37172.5	conv_dX_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
121420827	2186	55544.0	58143.5	12767	140343	43007.5	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
71802351	2186	30846.5	36975	17344	73087	15453.9	conv_forward_tiled_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
14739404	1874	7865.2	8464	2976	19775	4844.1	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
13911584	1874	7423.5	7664	1503	19616	5927.9	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
8203298	7496	1094.4	960	927	2752	290	sgd_update_kernel(float *, float *, int, float, int)
8099508	2811	2881.4	2752	1056	7008	1502.8	relu_backward_kernel(const float *, const float *, float *, int)
7704974	3279	2349.0	2304	959	5632	1142.5	relu_forward_kernel(const float *, float *, int)
6606758	2186	3022.3	3552	2208	5600	747.8	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
5246109	1874	2799.4	3136	2239	4864	502.4	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
4268859	1874	2277.9	2656	1727	4096	497.1	zero_kernel(float *, int)
3340635	1874	1782.6	1824	1631	2848	187.7	fc_backward_bias_kernel(const float *, float *, int, int)
1831469	937	1954.6	1920	1888	2944	175.4	softmax_cross_entropy_grad_kernel(const float *, const float *, float *, float *, int, int)
945557	937	1008.1	992	960	1472	86.5	scale_inplace_kernel(float *, int, float)

Figure 35: Batch_S : 64, Block_S : 512

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
72.2	4112330760	2811	1462942.3	22299.0	2438	6515594	2064458.8	cudaDeviceSynchronize
15.2	865867521	40	21646688.0	1903.5	877	865063163	136775240.2	cudaMalloc
5.9	337823054	4068	83044.0	33878.5	8477	959706	91716.6	cudaMemcpy
5.2	295986622	37010	7997.5	3773.0	2356	6434956	35447.4	cudaLaunchKernel
1.4	78305609	8433	9285.6	4303.0	2412	168785	10327.0	cudaMemset
0.1	3953093	37010	106.8	60.0	49	51499	891.1	cuKernelGetName
0.0	933138	40	23328.5	1675.5	925	264376	52534.6	cudaFree
0.0	456467	10	45646.7	49034.0	24296	61132	13737.7	cuLibraryLoadData
0.0	247811	10	24781.1	8611.5	6841	148074	43820.5	cuLibraryUnload
0.0	8192	1	8192.0	8192.0	8192	8192	0.0	cuModuleGetLoadingMode
0.0	3787	16	236.7	206.0	119	428	94.2	cuLibraryGetKernel
0.0	2324	1	2324.0	2324.0	2324	2324	0.0	cuCtxSynchronize
0.0	262	1	262.0	262.0	262	262	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\protobuf\src\util.cc:124: ParseError: Unexpected end of data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
73.2	19040347	2038	9342.7	16735.0	288	32704	8413.8	[CUDA memcpy Host-to-Device]
19.8	5134894	8433	608.9	352.0	287	4512	342.3	[CUDA memset]
7.0	1822768	2030	897.9	896.0	767	4160	164.6	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
386.115	8433	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.021	2038	0.108	0.201	0.000	0.401	0.100	[CUDA memcpy Host-to-Device]
2.802	2030	0.001	0.003	0.000	0.003	0.001	[CUDA memcpy Device-to-Host]

Figure 36: Batch_S : 64, Block_S : 512

1. GPU Active Time: 4196260 us
2. GPU Util : 73.6%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
  256
```

Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
270083284	15000	180045.6	197693	78527	1071955	108045.6	conv_dw_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int)
1118377091	15000	74558.5	89070.5	46911	629976	29267.6	conv_db_kernel(const float *, float *, int, int, int, int)
482850769	17500	27591.5	28047.5	3584	79488	23938.1	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
296201238	15000	19880.1	26048	12864	532986	10257.2	conv_dk_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int)
118997459	17500	6799.9	8864	3679	14720	2975.1	conv_forward_tiled_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
64016560	60000	1066.9	960	864	6816	306.4	sgd_update_kernel(float *, float *, int, float, int)
42179278	15000	2812	3183.5	1407	8064	1388.1	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
32059867	15000	2137.3	2383.5	1183	13632	946.9	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
27559948	26250	1049.9	1056	895	5408	123.1	relu_forward_kernel(const float *, float *, int)
26847193	17500	1534.1	1536	1439	5120	114.7	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
26439316	22500	1175.1	1152	1023	3040	128.7	relu_backward_kernel(const float *, const float *, float *, int)
20039006	15000	1336	1344	1215	54271	458.2	maxpool_backward_kernel(const float *, const int *, float *, int, int, int, int, int, int, int, int)
16064730	15000	1071	1056	1023	1664	72	fc_backward_bias_kernel(const float *, float *, int, int)
14565685	15000	971	992	832	2304	76.2	zero_kernel(float *, int)
13850086	7500	1846.7	1824	1760	2816	124.3	softmax_ce_loss_grad_kernel(const float *, const int *, float *, float *, int, int)
7278034	7500	970.4	960	927	1440	60.5	scale_inplace_kernel(float *, int, float)

Figure 37: Batch_S : 8, Block_S : 256

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
50.5	4814352829	22580	178415.6	9221.5	2135	2255512	244889.7	cudaDeviceSynchronize
27.0	2147368354	296250	7248.5	3880.0	2473	2229860	11157.2	cudaLaunchKernel
15.3	1216447582	32588	37419.9	14279.5	4259	731599	43243.6	cudaMemcpy
5.6	445073395	67500	6593.7	3879.0	2067	524869	8800.9	cudaMemset
1.3	108636718	40	2515918.0	1273.0	944	108496726	15889363.4	cudaMalloc
0.3	26090180	296250	88.1	58.0	49	194279	658.4	cuKernelGetName
0.0	446367	40	11159.2	1351.5	778	279654	46192.9	cudaFree
0.0	393543	10	39354.3	37975.5	24129	66161	12769.0	cuLibraryLoadData
0.0	231728	10	23172.8	10368.5	5404	112613	32882.3	cuLibraryUnload
0.0	12007	1	12007.0	12007.0	12007	12007	0.0	cuModuleGetLoadingMode
0.0	3488	1	3488.0	3488.0	3488	3488	0.0	cuCtxSynchronize
0.0	3425	16	214.1	203.0	94	399	90.6	cuLibraryGetKernel
0.0	588	1	588.0	588.0	588	588	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\Quad_Main\Imports\Source\ProtoBuf\protobuf-3.21.1\src\google\pr
ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
52.5	42969914	67500	636.6	704.0	287	19487	366.5	[CUDA memset]
30.6	25086848	16258	1543.0	2304.0	287	32608	1244.9	[CUDA memcpy Host-to-Device]
16.9	13796276	16250	849.0	832.0	767	13728	171.0	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
220.172	16258	0.014	0.025	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
2.830	16250	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 38: Batch_S : 8, Block_S : 256

1. GPU Active Time: 5002725 us
2. GPU Util : 62.9%

6.6 Optimization 3: Optimized Backward Pass with Atomic Re-duction

6.6.1 Problem: Atomic Contention

Computing weight gradients requires accumulating contributions from all batch samples:

$$\frac{\partial L}{\partial W[c_{out}, c_{in}, kh, kw]} = \sum_{b=0}^{B-1} \sum_h \sum_w \frac{\partial L}{\partial Y[b, c_{out}, h, w]} \cdot X[b, c_{in}, \dots] \quad (28)$$

Naive implementation: Each thread atomically adds to shared weight gradient:

```

1 __global__ void conv_backward_weights_naive(
2     const float* dY,      // Gradient from output
3     const float* X,       // Input
4     float* dW)            // Weight gradients (output)
5 {
6     // Each thread processes one (b, h, w) position
7     int idx = blockIdx.x * blockDim.x + threadIdx.x;
8
9     // Compute which weight position contributes
10    int w_idx = compute_weight_index(...);
11    float grad = dY[idx] * X[...];
12
13    // PROBLEM: Many threads atomically add to same location!
14    atomicAdd(&dW[w_idx], grad); // Serialization!
15 }

```

Listing 10: Naive Atomic Approach - High Contention

Number of atomic operations:

- Batch size: $B = 64$
- Output spatial size: $28 \times 28 = 784$
- Total atomics per weight: $64 \times 784 = 50,176$ atomic operations!

With thousands of weights, this creates massive contention and serialization.

6.6.2 Solution: Two-Level Reduction

Level 1: Per-block reduction in shared memory

```

1 __global__ void conv_backward_weights_optimized(
2     const float* dY,
3     const float* X,
4     float* dW)
5 {
6     __shared__ float shared_grad[256]; // One per thread
7
8     int tid = threadIdx.x;
9     int idx = blockIdx.x * blockDim.x + tid;
10
11    // Each thread computes partial gradient
12    float my_grad = 0.0f;
13    for (int i = idx; i < total_work; i += gridDim.x * blockDim.x
        ) {

```

```

14         my_grad += dY[i] * X[...];
15     }
16     shared_grad[tid] = my_grad;
17     __syncthreads();
18
19     // Tree reduction in shared memory
20     for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
21         if (tid < stride) {
22             shared_grad[tid] += shared_grad[tid + stride];
23         }
24         __syncthreads();
25     }
26
27     // Only thread 0 does atomic add (one per block!)
28     if (tid == 0) {
29         atomicAdd(&dW[weight_idx], shared_grad[0]);
30     }
31 }

```

Listing 11: Block-Level Reduction (Tree Reduction)

Level 2: Per-grid atomic (one per block)

After block-level reduction, only one thread per block performs atomic operation.

6.6.3 Performance Improvement

Atomic Operation Reduction:

- **Before:** $B \times H \times W = 64 \times 784 = 50,176$ atomics per weight
- **After:** num_blocks 512 atomics per weight
- **Reduction:** $50,176 / 512$ **98× fewer atomics!**

6.7 Optimization 4: Stream-Based Data Transfer Overlapping

6.7.1 Problem: CPU-GPU Transfer Latency

In sequential execution:

```

1 for each batch:
2     1. Prepare batch on CPU           (2 ms)
3     2. Transfer CPU      GPU (H2D)    (4 ms)
4     3. Execute GPU kernels            (16 ms)
5     4. Transfer GPU      CPU (D2H)     (1 ms)
6
7 Total: 2 + 4 + 16 + 1 = 23 ms per batch

```

Listing 12: Sequential Execution (Inefficient)

GPU is idle during steps 1, 2, and 4 (30% idle time!).

6.7.2 Solution: Concurrent Execution with Streams

Use two CUDA streams to overlap operations:

```
1 // Create two streams
2 cudaStream_t stream[2];
3 cudaStreamCreate(&stream[0]);
4 cudaStreamCreate(&stream[1]);
5
6 // Allocate ping-pong buffers
7 float* d_X[2]; // Two device buffers
8 cudaMalloc(&d_X[0], size);
9 cudaMalloc(&d_X[1], size);
10
11 // Pinned host memory for async transfer
12 float* h_X_pinned[2];
13 cudaMallocHost(&h_X_pinned[0], size);
14 cudaMallocHost(&h_X_pinned[1], size);
15
16 for (int step = 0; step < num_steps; step++) {
17     int current = step % 2;
18     int next = (step + 1) % 2;
19
20     // Transfer NEXT batch (async on stream[next])
21     if (step + 1 < num_steps) {
22         prepare_batch(h_X_pinned[next], step + 1);
23         cudaMemcpyAsync(d_X[next], h_X_pinned[next], size,
24                        cudaMemcpyHostToDevice, stream[next]);
25     }
26
27     // Process CURRENT batch (on stream[current])
28     kernels<<<grid, block, 0, stream[current]>>>(d_X[current],
29     ...);
30 }
```

Listing 13: Stream-Based Overlapping

6.8 Profiling:

In this section we will profile different configuration for block size and batch size that may effect our performance :

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte          --train-labels train-labels.idx1-ubyte          --
  test-images t10k-images.idx3-ubyte          --test-labels t10k-
  labels.idx1-ubyte          --epochs 1 --batch 64 --lr 0.01 --
  block 256
```


Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
38.0	428573287	33732	12468.1	9668.0	2720	429076	18460.5	cudaLaunchKernel
28.0	309265324	2820	109668.6	21786.5	3384	1489683	180555.2	cudaMemcpyAsync
25.9	286140858	40	7153521.5	1739.5	848	285260163	45100102.8	cudaMalloc
6.2	68401539	8433	8111.2	6665.0	1901	194726	9615.5	cudaMemsetAsync
0.7	7369774	937	7865.3	7608.0	3271	38250	4211.0	cudaStreamSynchronize
0.4	3914824	33732	116.1	109.0	50	29446	396.7	cuKernelGetName
0.3	3277472	1876	1747.1	1618.5	583	29312	1374.7	cudaEventRecord
0.2	2531155	1874	1350.7	1191.5	431	16809	913.7	cudaStreamWaitEvent
0.1	1581514	40	39537.8	2422.5	1318	375668	91664.8	cudaFree
0.1	850654	4	212663.5	15986.0	3551	815131	401758.4	cudaHostAlloc
0.1	692770	4	173192.5	10579.5	3309	668302	330108.1	cudaFreeHost
0.1	658712	10	65871.2	62856.5	38553	98839	17755.4	cuLibraryLoadData
0.0	347966	8	43495.8	24637.0	10043	167934	53698.8	cudaMemcpy
0.0	272418	10	27241.8	13559.5	7879	107985	32721.8	cuLibraryUnload
0.0	74159	2	37079.5	37079.5	1868	72251	49796.6	cudaStreamCreate
0.0	22221	1	22221.0	22221.0	22221	22221	0.0	cudaDeviceSynchronize
0.0	12616	1	12616.0	12616.0	12616	12616	0.0	cuModuleGetLoadingMode
0.0	10733	2	5366.5	5366.5	1159	9574	5950.3	cudaStreamDestroy
0.0	10221	4	2555.3	606.0	479	8530	3984.0	cudaEventCreateWithFlags
0.0	5692	16	355.8	332.5	261	567	85.4	cuLibraryGetKernel
0.0	5606	4	1401.5	730.0	327	3819	1633.7	cudaEventDestroy
0.0	2912	1	2912.0	2912.0	2912	2912	0.0	cuCtxSynchronize
0.0	427	1	427.0	427.0	427	427	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\pr
ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
75.9	21100314	1882	11211.6	3520.0	416	34304	9537.0	[CUDA memcopy Host-to-Device]
21.1	5863738	8433	695.3	736.0	287	5920	368.2	[CUDA memset]
3.0	830676	946	878.1	832.0	768	1696	90.1	[CUDA memcopy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
386.115	8433	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
188.712	1882	0.100	0.000	0.000	0.401	0.100	[CUDA memcopy Host-to-Device]
0.027	946	0.000	0.000	0.000	0.003	0.000	[CUDA memcopy Device-to-Host]

Figure 39: Batch_S : 64, Block_S : 256

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
28.5	133890952	1874	71449.8	95583	29440	686904	49441	conv_dk_kernel_v1(const float *, const float *, float *, int, int, int, int, int, int, int, int)
17.9	8994202	1874	44793.6	46367	26559	735480	35842.7	conv_dk_kernel_v2_1(const float *, const float *, float *, int, int, int, int, int, int, int, int)
14.3	67116642	1874	35814.6	38703.5	17408	74815	17427.3	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int)
13.4	60942848	1874	32520.2	31359.5	7008	80383	25336.9	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
11.4	53549029	1874	28574.7	29103.5	11167	61183	16935.1	conv_db_reduce_kernel_v2(const float *, float *, int, int, int, int)
3.2	14895073	1874	7948.5	8352	2527	23360	5304.4	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
3.1	14560051	1874	7769.5	8831.5	1503	19039	6297.3	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
1.8	8656107	2811	3079.4	2688	1055	12736	1766.1	relu_backward_kernel(const float *, const float *, float *, int)
1.8	8640784	7496	1152.7	960	895	9504	388.5	sgd_update_kernel(float *, float *, int, float, int)
1.3	5938964	1874	3169.1	3392	2272	10048	805.9	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
1.2	5661105	1874	3020.9	3072	2240	38399	1118.2	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
1	4569647	1874	2438.4	2656	1696	13952	669.8	zero_kernel(float *, int)
0.8	3599891	1874	1921	1824	1631	3072	299.8	fc_backward_bias_kernel(const float *, float *, int, int)
0.4	2010408	937	2145.6	1952	1887	3200	327.6	softmax_loss_grad_kernel(const float *, const float *, float *, float *, int, int)
0.2	988977	937	1055.5	992	927	1472	145.5	relu_forward_kernel(const float *, float *, int)
0.2	967895	937	1054.3	960	928	1440	144.9	scale_inplace_kernel(float *, int, float)

Figure 40: Batch_S : 64, Block_S : 256

1. GPU Active Time: 428901 us
2. GPU Util : 38.9%

```
nsys profile --stats=true ./cnn --train-images train-images.idx3-
ubyte --train-labels train-labels.idx1-ubyte --
test-images t10k-images.idx3-ubyte --test-labels t10k-
labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
512
```


Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
47.7	2205847863	270000	8169.8	3530.0	2550	2525695	16580.1	cudaLaunchKernel
28.0	1296948654	22575	57410.8	9823.0	3006	1165729	92483.1	cudaMemcpyAsync
15.6	721180199	40	18029505.0	1221.5	988	721097952	114015263.8	cudaMalloc
6.7	308821294	67500	4575.1	3413.0	1807	233345	5029.7	cudaMemsetAsync
0.8	34779108	7500	4637.2	3445.0	1843	140360	4430.2	cudaStreamSynchronize
0.5	25237322	270000	93.5	59.0	49	136467	884.8	cuKernelGetName
0.4	18585149	15002	1238.8	844.0	470	99461	2325.8	cudaEventRecord
0.3	15242099	15000	1016.1	643.0	382	95926	2256.5	cudaStreamWaitEvent
0.0	555209	4	138802.3	5254.5	2910	541790	268667.1	cudaHostAlloc
0.0	412955	10	41295.5	35987.0	25171	69074	14319.8	cuLibraryLoadData
0.0	369096	4	92274.0	6913.5	1840	353429	174148.5	cudaFreeHost
0.0	290158	8	36269.8	21761.0	9535	123238	39120.1	cudaMemcpy
0.0	220817	10	22081.7	11083.0	6711	111564	31956.1	cuLibraryUnload
0.0	168071	40	4201.8	1089.5	803	114355	17907.6	cudaFree
0.0	48787	2	24393.5	24393.5	846	47941	33301.2	cudaStreamCreate
0.0	12688	1	12688.0	12688.0	12688	12688	0.0	cudaDeviceSynchronize
0.0	9274	1	9274.0	9274.0	9274	9274	0.0	cuModuleGetLoadingMode
0.0	7087	4	1771.8	459.0	354	5815	2696.2	cudaEventCreateWithFlags
0.0	6171	2	3085.5	3085.5	682	5489	3399.1	cudaStreamDestroy
0.0	4417	4	1104.3	464.0	285	3204	1408.2	cudaEventDestroy
0.0	3780	16	236.3	238.0	139	390	71.1	cuLibraryGetKernel
0.0	2551	1	2551.0	2551.0	2551	2551	0.0	cuCtxSynchronize
0.0	367	1	367.0	367.0	367	367	0.0	cuDeviceGetUuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agona\Rel\Quad_Main\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\protobuf\api.proto:100:1: data when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
49.6	45639025	67500	676.1	768.0	287	25792	349.9	[CUDA memset]
43.3	39832840	15008	2654.1	3296.0	416	33280	1514.8	[CUDA memcpy Host-to-Device]
7.2	6607361	7575	872.3	864.0	768	4288	81.7	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
188.812	15008	0.013	0.000	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
0.054	7575	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 41: $Batch_S : 8, Block_S : 512$

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
46.8	705747919	15000	51049.9	51615.5	9552	149214	47758.4	fc_forward_kernel(const float *, const float *, float *, int, int, int)
20	326727280	15000	21781.8	25055.5	14336	43136	7358.8	conv_dk_kernel_v1(const float *, const float *, float *, int, int, int, int, int, int, int)
6.8	111712733	15000	7447.5	8128	6336	17280	1129.3	conv_dw_partial_kernel_v2_k3(const float *, const float *, float *, int, int, int, int, int, int, int)
6.3	103110590	15000	6874	9391.5	3679	17023	3054.5	conv_forward_jitted_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
4	65021005	60000	1083.7	960	896	2752	290.9	spt_update_kernel(float *, float *, int, float, int)
3.2	52137959	15000	3475.9	4384	2335	8832	1125	conv_db_reduce_kernel_v2(const float *, float *, int, int, int, int)
2.9	47197269	15000	3146.5	3823.5	1471	8896	1642.6	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
2.1	33979887	15000	2265.3	2943.5	1247	8032	980.6	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
1.7	27190368	22500	1208.5	1152	1056	3904	136.1	relu_backward_kernel(const float *, const float *, float *, int)
1.5	24239552	15000	1616	1600	1536	5600	153.4	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
1.3	21209135	15000	1413.9	1376	1312	54527	494.4	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
1	16170830	15000	1078.1	1056	1023	1664	96.4	fc_backward_bias_kernel(const float *, float *, int, int)
0.9	13927958	7500	1857.1	1824	1728	2944	170.3	softmax_cross_entropy_loss_grad_kernel(const float *, const float *, float *, float *, int, int)
0.9	13919491	15000	928	928	932	3104	87.1	zero_kernel(float *, int)
0.5	7432365	7500	991	960	959	1472	85.4	relu_forward_kernel(const float *, float *, int)
0.4	7337025	7500	978.3	960	927	1440	83.5	scale_inplace_kernel(float *, int, float)

Figure 42: $Batch_S : 8, Block_S : 512$

1. GPU Active Time: 1584978 us
2. GPU Util : 34.25%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-ubyte
  --train-labels train-labels.idx1-ubyte --test-images t10k-images.idx3-ubyte
  --test-labels t10k-labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block 1024
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
63.7	3162796434	22575	140101.7	7674.0	3229	1256807	202944.3	cudaMemcpyAsync
25.8	1281176596	27000	4745.1	3329.0	2495	421236	6942.7	cudaLaunchKernel
6.0	298216095	67500	4418.0	3387.0	1835	361820	4312.6	cudaMemsetAsync
2.9	145572645	40	3639316.1	1162.0	960	145480047	23002040.4	cudaMalloc
0.7	33206528	7500	4427.5	3469.0	2006	92216	3830.6	cudaStreamSynchronize
0.4	18939153	27000	70.1	58.0	48	36883	263.2	cuKernelGetName
0.3	15388313	15002	1025.8	844.0	472	86086	1407.0	cudaEventRecord
0.2	11453892	15000	763.6	660.0	392	30277	750.0	cudaStreamWaitEvent
0.0	479726	4	119931.5	3231.5	2173	471090	234107.4	cudaHostAlloc
0.0	403310	10	40331.0	33752.0	22204	64356	15785.9	cuLibraryLoadData
0.0	402482	4	100620.5	6797.0	2062	386826	190838.0	cudaFreeHost
0.0	391811	10	39181.1	14599.5	7920	135622	44090.5	cuLibraryUnload
0.0	300681	8	37585.1	20268.0	10462	131128	41687.8	cudaMemcpy
0.0	192900	40	4822.5	1408.0	821	132762	20779.8	cudaFree
0.0	52339	2	26169.5	26169.5	1104	51235	35448.0	cudaStreamCreate
0.0	14117	1	14117.0	14117.0	14117	14117	0.0	cudaDeviceSynchronize
0.0	8944	1	8944.0	8944.0	8944	8944	0.0	cuModuleGetLoadingMode
0.0	7804	4	1951.0	380.5	226	6817	3245.1	cudaEventCreateWithFlags
0.0	6321	2	3160.5	3160.5	645	5676	3557.5	cudaStreamDestroy
0.0	3787	4	926.8	449.5	199	2609	1137.9	cudaEventDestroy
0.0	3149	1	3149.0	3149.0	3149	3149	0.0	cuCtxSynchronize
0.0	3129	16	195.6	159.5	96	496	98.3	cuLibraryGetKernel
0.0	275	1	275.0	275.0	275	275	0.0	cuDeviceGetKernel

[5/7] Executing 'cuda_gpu_kern_sum' stats report
libprotobuf ERROR C:\dvs\p4\build\sw\devtools\Agora\Rel\Quad_Main\Imports\Source\ProtoBuf\protobuf-3.21.1\src\google\proto
sta when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
40.7	45697559	67500	677.0	768.0	207	29983	355.2	[CUDA memset]
44.5	41735820	15000	2780.9	3408.0	416	33247	1600.6	[CUDA memcpy Host-to-Device]
6.8	6392175	7575	843.9	832.0	768	14144	177.7	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
188.812	15000	0.013	0.000	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
0.054	7575	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 43: Batch_S : 8, Block_S : 1024

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
58.3	1478772089	15000	99584.8	97774.5	3647	295421	95492	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
16.3	412401928	15000	27498.8	32608	10464	54943	8957.1	conv_dk_kernel_v1(const float *, const float *, float *, int, int, int, int, int, int, int)
6.3	159050343	15000	10603.4	14055	6143	66865	4433.1	conv_dw_partial_kernel_v2_16(const float *, const float *, float *, int, int, int, int, int, int, int)
4.1	102904045	15000	6860.3	7839.5	3648	14720	3035.3	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
2.9	72342418	15000	4822.8	5711.5	1599	12544	3182.3	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
2.8	69839568	60000	1164	992	927	8000	419.1	sgd_update_kernel(float *, float *, int, float, int)
2	51592308	15000	3430.5	4064	2623	7456	783.4	conv_dw_reduce_kernel_v2(const float *, float *, int, int, int, int, int, int, int, int)
1.7	42630291	15000	2842	3655.5	1530	7776	1261	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int, int)
1.2	30219667	22500	1343.1	1184	1119	5472	255	relu_backward_kernel(const float *, const float *, float *, int)
1.1	28092183	15000	1912.8	1888	1824	4287	153.3	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
1	24986725	15000	1665.8	1632	1568	22304	228.8	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
0.6	16352349	15000	1090.2	1088	1023	1696	83.0	fc_backward_bias_kernel(const float *, float *, int, int)
0.6	15523748	15000	1034.9	1120	832	2912	150.1	zero_kernel(float *, int)
0.6	13981250	7500	1864.2	1824	1760	24928	301.7	softmax_cross_entropy_loss_grad_kernel(const float *, const float *, float *, float *, int, int, int)
0.3	7757057	7500	1034.3	1024	991	2720	79.3	relu_forward_kernel(const float *, float *, int)
0.3	7488013	7500	995.8	992	959	1472	71.9	scale_inplace_kernel(float *, int, float)

Figure 44: Batch_S : 8, Block_S : 1024

1. GPU Active Time: 2519363 us
2. GPU Util : 50.7%

```
1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 8 --lr 0.01 --block
  256
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
57.7	2269919870	270000	8407.1	3735.0	2526	866691	15716.0	cudaLaunchKernel
24.2	951674711	22575	42156.1	5507.0	2972	1281268	82284.7	cudaMemcpyAsync
9.3	364268406	67500	5396.6	3171.0	1825	241453	9137.3	cudaMemsetAsync
6.9	271535122	40	6788378.0	1552.0	1018	271436207	42917431.9	cudaMalloc
0.8	36058032	7500	4807.7	3142.5	1909	183529	4442.9	cudaStreamSynchronize
0.5	19004104	270000	70.4	58.0	47	116985	443.0	cudaKernelGetName
0.4	14190668	15002	945.9	704.5	424	688683	6189.0	cudaEventRecord
0.3	11011036	15000	734.1	577.0	409	226679	2493.9	cudaStreamWaitEvent
0.0	885075	4	201268.8	3009.5	1901	797155	397258.1	cudaHostAlloc
0.0	585321	4	146330.3	8536.5	3322	564926	279093.9	cudaFreeHost
0.0	535994	2	267997.0	267997.0	1193	534801	377317.8	cudaStreamCreate
0.0	513952	10	51395.2	44320.0	27952	82656	22160.7	cuLibraryLoadData
0.0	328703	8	41087.9	24082.0	10987	143400	45282.0	cudaMemcpy
0.0	288330	4	72082.5	556.5	258	286959	143251.2	cudaEventCreateWithFlags
0.0	278634	40	6965.9	1298.5	838	159869	25825.8	cudaFree
0.0	227566	10	22756.6	10274.0	7608	106940	31349.4	cuLibraryUnload
0.0	16745	1	16745.0	16745.0	16745	16745	0.0	cudaDeviceSynchronize
0.0	8959	2	4479.5	4479.5	947	8012	4995.7	cudaStreamDestroy
0.0	8702	1	8702.0	8702.0	8702	8702	0.0	cuModuleGetLoadingMode
0.0	4728	4	1182.0	367.0	319	3675	1662.4	cudaEventDestroy
0.0	4272	16	267.0	263.5	170	588	102.3	cuLibraryGetKernel
0.0	2448	1	2448.0	2448.0	2448	2448	0.0	cuCtxSynchronize
0.0	396	1	396.0	396.0	396	396	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\ip4\build\sw\devtools\Agora\Rel\QuadMain\Imports\Source\ProtoBuf\protobuf-3_21_1\src\google\protobuf\ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
48.2	46876592	67500	694.5	768.0	287	29343	352.5	[CUDA memset]
45.1	43906180	15008	2925.5	3712.0	416	32352	1722.5	[CUDA memcpy Host-to-Device]
6.7	6480563	7575	855.5	832.0	768	4640	89.2	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
3090.570	67500	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
188.812	15008	0.013	0.000	0.000	0.401	0.013	[CUDA memcpy Host-to-Device]
0.054	7575	0.000	0.000	0.000	0.000	0.000	[CUDA memcpy Device-to-Host]

Figure 45: $Batch_S : 8, Block_S : 256$

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
33	423815215	15000	28254.3	28006	3616	80319	24836.3	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
23.7	304219071	15000	20281.3	22847.5	12063	40704	7406.5	conv_dw_kernel_v1(const float *, const float *, float *, int, int, int, int, int, int, int, int)
8.8	113087943	15000	7539.2	8512	5983	43808	1507.7	conv_dw_partial_kernel_v2_4(const float *, const float *, float *, int, int, int, int, int, int, int, int)
8.1	104689913	15000	6979.3	8672	3648	20864	3163.7	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
5.4	68810021	15000	4587.3	6207	2623	10592	1935.9	conv_db_reduce_kernel_v2(const float *, float *, int, int, int, int)
5	64511100	60000	1075.2	960	895	10016	314.8	sgd_update_kernel(float *, float *, int, float, int)
3.3	42734014	15000	2848.9	3344	1375	7080	1429.6	backward_input_kernel(const float *, const float *, float *, int, int, int)
2.6	33417654	15000	2227.8	2960	1151	5024	1044.6	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
2.1	27079727	22500	1203.5	1184	992	4416	162.3	relu_backward_kernel(const float *, const float *, float *, int)
1.8	23169978	15000	1544.7	1504	1407	4064	189.1	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
1.6	20266546	15000	1351.1	1344	1184	25444	324.8	maxpool_backward_kernel(const float *, const float *, const float *, float *, int, int, int, int, int, int, int)
1.3	16284336	15000	1085.6	1056	1023	4512	125.1	fc_backward_bias_kernel(const float *, float *, int, int, int)
1.2	14790483	15000	986	992	863	2912	116.9	zero_kernel(float *, int)
1.1	13951904	7500	1860.3	1792	1759	2816	211.7	softmax_loss_grad_kernel(const float *, const float *, float *, float *, int, int)
0.6	7415382	7500	988.7	960	927	1440	107	scale_inplace_kernel(float *, int, float)
0.6	7083382	7500	944.5	928	895	1408	101.9	relu_forward_kernel(const float *, float *, int)

Figure 46: $Batch_S : 8, Block_S : 256$

1. GPU Active Time: 1270796 us
2. GPU Util : 32.2%

```
nsys profile --stats=true ./cnn --train-images train-images.idx3-ubyte
--train-labels train-labels.idx1-ubyte --test-images t10k-images.idx3-ubyte
--test-labels t10k-labels.idx1-ubyte --epochs 1 --batch 32 --lr 0.01 --block 256
```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
48.8	624087511	67500	9245.7	3783.0	2599	308897	19371.2	cudaLaunchKernel
36.7	468808571	5643	83076.5	9975.0	3240	1014896	134114.9	cudaMemcpyAsync
7.0	88846155	40	2221153.9	1553.0	975	88655681	14016829.1	cudaMalloc
5.9	74831533	16875	4434.5	3375.0	1911	218873	6244.5	cudaMemsetAsync
0.6	7645197	1875	4077.4	3559.0	2240	41021	2122.6	cudaStreamSynchronize
0.4	4767700	67500	70.6	59.0	50	23689	276.8	cuKernelGetName
0.3	3630751	3752	967.7	881.0	495	21458	801.0	cudaEventRecord
0.2	3001459	3750	800.4	701.0	413	92773	1595.8	cudaStreamWaitEvent
0.0	563068	40	14076.7	1405.5	965	189707	44235.1	cudaFree
0.0	506588	4	126647.0	6883.5	2275	490546	242625.0	cudaFreeHost
0.0	454368	4	113592.0	2512.0	1681	447663	222714.3	cudaHostAlloc
0.0	368686	10	36868.6	35777.5	20191	61200	12568.7	cuLibraryLoadData
0.0	335710	8	42463.0	20565.0	10095	175791	55754.5	cudaMemcpy
0.0	230499	10	23049.9	12024.0	8152	112168	31487.0	cuLibraryUnload
0.0	59049	2	29524.5	29524.5	1203	57846	40052.6	cudaStreamCreate
0.0	15035	1	15035.0	15035.0	15035	15035	0.0	cudaDeviceSynchronize
0.0	9745	1	9745.0	9745.0	9745	9745	0.0	cuModuleGetLoadingMode
0.0	6681	4	1670.3	321.5	287	5751	2720.6	cudaEventCreateWithFlags
0.0	6113	2	3056.5	3056.5	663	5450	3384.9	cudaStreamDestroy
0.0	4050	16	253.1	182.5	124	1079	233.0	cuLibraryGetKernel
0.0	3293	4	823.3	399.0	201	2294	990.5	cudaEventDestroy
0.0	3082	1	3082.0	3082.0	3082	3082	0.0	cuCtxSynchronize
0.0	238	1	238.0	238.0	238	238	0.0	cuDeviceGetLuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\dvs\pal\buildaw\devtools\Agora\Rel\Quand_Main\Imports\Source\Protobuf\protobuf-3_21_1\src\google\proto] ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
62.0	22553714	3758	6001.5	2528.0	416	32704	4427.3	[CUDA memcpy Host-to-Device]
33.5	12176259	16875	721.6	768.0	287	1856	360.4	[CUDA memset]
4.5	1652325	1893	872.9	832.0	768	1376	66.7	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
772.643	16875	0.046	0.000	0.000	0.401	0.125	[CUDA memset]
188.812	3758	0.050	0.000	0.000	0.401	0.050	[CUDA memcpy Host-to-Device]
0.031	1893	0.000	0.000	0.000	0.001	0.000	[CUDA memcpy Device-to-Host]

Figure 47: $Batch_S : 32, Block_S : 256$

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
25.6	152331903	3750	40621.8	41071.5	16959	86399	23469.5	conv_dk_kernel_v1(const float *, const float *, float *, int, int, int, int, int, int, int, int, int, int)
20.5	121786308	3750	32476.3	31119.5	6975	80319	25840.4	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
14.5	86463051	3750	23057	24911.5	14912	40096	7754.7	conv_dw_partial_kernel_v2_K3(const float *, const float *, float *, int, int, int, int, int, int, int, int)
12.5	74207104	3750	19809.9	21311.5	9440	40895	10109	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
9.5	56725993	3750	15126.9	15295.5	6271	31584	8734.4	conv_db_reduce_kernel_v2(const float *, float *, int, int, int)
3.5	20622231	3750	5499.3	6111.5	1407	12832	4026.5	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
2.9	17384470	3750	4635.9	5503.5	1759	14688	2825.1	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
2.9	17301543	15000	1153.4	960	896	7584	368.7	sgd_update_kernel(float *, float *, int, float, int)
1.9	11557627	5625	2054.7	1824	1024	4160	850.6	relu_backward_kernel(const float *, const float *, float *, int)
1.4	8434142	3750	2249.1	2304	1696	10047	562.2	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
1.3	8001045	3750	2133.6	2176	1600	3264	444.2	maxpool_backward_kernel(const float *, const int *, float *, int, int, int, int, int, int, int, int)
1.1	6267461	3750	1671.3	1728	1248	2624	387.6	zero_kernel(float *, int)
0.9	5487578	3750	1463.4	1344	1247	2112	273.4	fc_backward_bias_kernel(const float *, float *, int, int)
0.7	3891506	1875	2075.5	1888	1855	2912	380.3	softmax_cross_entropy_loss_grad_kernel(const float *, const int *, float *, float *, int, int)
0.3	1988654	1875	1060.6	960	928	1440	184.1	scale_inplace_kernel(float *, int, float)
0.3	1930025	1875	1029.3	928	896	1440	178.8	relu_forward_kernel(const float *, float *, int)

Figure 48: $Batch_S : 32, Block_S : 256$

1. GPU Active Time: 590936 us
2. GPU Util : 46.2%

```

1 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte --train-labels train-labels.idx1-ubyte --
  test-images t10k-images.idx3-ubyte --test-labels t10k-
  labels.idx1-ubyte --epochs 1 --batch 64 --lr 0.01 --
  block 512

```

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
50.7	754044498	40	18851112.4	2011.0	1088	753539322	119142984.0	cudaMalloc
33.9	503296517	2820	178473.9	12862.0	3138	1209854	256995.4	cudaMemcpyAsync
12.3	183155687	33732	5429.7	3692.0	2669	418945	10550.1	cudaLaunchKernel
2.3	33711640	8433	3997.6	3329.0	1829	56923	3800.5	cudaMemsetAsync
0.3	3764651	937	4017.8	3686.0	2550	30383	1551.8	cudaStreamSynchronize
0.2	2355507	33732	69.8	59.0	49	20596	226.1	cuKernelGetName
0.1	1794850	1876	956.7	880.0	523	17078	631.4	cudaEventRecord
0.1	1636288	1874	873.2	785.5	440	12889	544.0	cudaStreamWaitEvent
0.1	840875	40	21001.9	1491.0	790	164981	46636.2	cudaFree
0.0	59029	4	137582.3	2703.5	1938	542804	270268.1	cudaHostAlloc
0.0	483723	8	60465.4	64751.5	13440	142410	38033.0	cudaMemcpy
0.0	482549	4	100662.3	6753.0	2179	386064	100900.2	cudaFreeHost
0.0	371892	10	37189.2	32919.5	18594	60398	14552.2	cuLibraryLoadData
0.0	210201	10	21020.1	11142.0	6757	103505	29506.0	cuLibraryUnload
0.0	34081	2	17040.5	17040.5	885	33196	22847.3	cudaStreamCreate
0.0	15602	1	15602.0	15602.0	15602	15602	0.0	cudaDeviceSynchronize
0.0	9051	1	9051.0	9051.0	9051	9051	0.0	cuModuleGetLoadingMode
0.0	6926	4	1731.5	387.5	265	5886	2771.7	cudaEventCreateWithFlags
0.0	6763	2	3381.5	3381.5	916	5847	3486.7	cudaStreamDestroy
0.0	4157	4	1039.3	469.0	313	2906	1246.9	cudaEventDestroy
0.0	3370	16	210.6	181.5	108	535	104.7	cuLibraryGetKernel
0.0	2928	1	2928.0	2928.0	2928	2928	0.0	cuCtxSynchronize
0.0	256	1	256.0	256.0	256	256	0.0	cuDeviceGetUuid

[5/7] Executing 'cuda_gpu_kern_sum' stats report
[libprotobuf ERROR C:\devs\p4\build\sw\devtools\Agona\Rel\QuadD_Main\Imports\Source\ProtoBuf\protobuf-3.21.1\src\google\pr] ata when parsing a protocol buffer. Use the 'bytes' type if you intend to send raw bytes.
[6/7] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
73.9	19186652	1882	10194.8	2736.0	416	32608	8315.6	[CUDA memcpy Host-to-Device]
22.9	5955609	8433	706.2	736.0	287	1984	382.9	[CUDA memcpy]
3.2	826549	946	873.7	832.0	800	1280	62.8	[CUDA memcpy Device-to-Host]

[7/7] Executing 'cuda_gpu_mem_size_sum' stats report

Total (MB)	Count	Avg (MB)	Med (MB)	Min (MB)	Max (MB)	StdDev (MB)	Operation
386.115	8433	0.046	0.000	0.000	0.401	0.126	[CUDA memset]
188.712	1882	0.100	0.000	0.000	0.401	0.100	[CUDA memcpy Host-to-Device]
0.027	946	0.000	0.000	0.000	0.003	0.000	[CUDA memcpy Device-to-Host]

Figure 49: Batch_S : 64, Block_S : 512

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
27.9	139486402	1874	74432.4	71919	28448	154494	44012.1	conv_dk_kernel_v1(const float *, const float *, float *, int, int, int, int, int, int, int, int)
23.7	11864709	1874	63312.6	58111	12767	149022	50466.7	fc_forward_kernel(const float *, const float *, const float *, float *, int, int, int)
14	7004111	1874	37376.8	36943.5	17408	72959	18600.6	conv_forward_tiled_fused_kernel(Tensor, Tensor, const float *, Tensor, int, int, int)
12.3	61598824	1874	32870.2	31296	25727	48000	6374.5	conv_dw_partial_kernel_v2_k3(const float *, const float *, float *, int, int, int, int, int, int, int, int)
6.8	34114131	1874	18203.9	17631.5	7264	36992	10372.6	conv_db_reduce_kernel_v2(const float *, float *, int, int, int, int)
3.4	16831439	1874	9034.9	8512	3040	19360	5775.5	fc_backward_weight_kernel(const float *, const float *, float *, int, int, int)
3.1	15602085	1874	8325.6	7940	1503	19552	6795.5	fc_backward_input_kernel(const float *, const float *, float *, int, int, int)
1.8	9137885	7496	1219	1024	896	2720	365.9	sgd_update_kernel(float *, float *, int, float, int)
1.8	9005648	2811	3225.1	2784	1056	6912	1749.7	relu_backward_kernel(const float *, const float *, float *, int)
1.3	6514977	1874	3476.5	3584	2304	5632	962.6	maxpool_forward_kernel(const float *, float *, int *, int, int, int, int, int, int, int, int)
1.2	5852229	1874	3122.9	3168	2208	4032	708.4	maxpool_backward_kernel(const float *, const float *, float *, int, int, int, int, int, int, int, int)
1	4789221	1874	2555.6	2639.5	1696	4032	658.4	zero_kernel(float *, int)
0.8	3775147	1874	2014.5	1856	1632	2816	349.2	fc_backward_bias_kernel(const float *, float *, int, int)
0.4	2058061	937	2196.4	1920	1887	2944	359.8	softmax_cross_entropy_loss_grad_kernel(const float *, const float *, float *, int, int)
0.2	1052777	937	1123.6	992	959	1472	175.9	scale_inplace_kernel(float *, int, float)
0.2	1023894	937	1092.7	992	928	1440	169.6	relu_forward_kernel(const float *, float *, int)

Figure 50: Batch_S : 64, Block_S : 512

1. GPU Active Time: 456422 us
2. GPU Util : 30%

```
nsys profile --stats=true ./cnn --train-images train-images.idx3-ubyte
--train-labels train-labels.idx1-ubyte --test-images t10k-images.idx3-ubyte
--test-labels t10k-labels.idx1-ubyte --epochs 1 --batch 64 --lr 0.01 --block 512
```

7 Performance Analysis and Results

7.1 Test Parameters

Parameter	Values Tested
Batch Sizes	8,32,64
Block Sizes (threads)	256 ,512 ,1024
Tile Sizes	8×8 , 16×16
Epochs	14

Table 3: Experimental Parameters

7.1.1 Profiling Tools Usage

Nsight Compute (Kernel Analysis):

```
1 # Generate timeline trace
2 nsys profile --stats=true ./cnn --train-images train-images.idx3-
  ubyte          --train-labels train-labels.idx1-ubyte          --
  test-images t10k-images.idx3-ubyte          --test-labels t10k-
  labels.idx1-ubyte          --epochs 1 --batch 64 --lr 0.01 --
  block 256
```

7.2 Profiling Summary Table

In this section we will compare GPU utilization and GPU active times baseline and each optimization version that we implemented.

7.2.1 Definition of GPU Active Time

In this work, GPU Active Time is defined as the total kernel execution time, i.e., the sum of all CUDA kernel durations reported by Nsight Systems. This metric reflects pure computational performance and excludes host-device memory transfers and CPU-side preprocessing.

Version	Batch	Block	Tile	GPU Util (%)	Kernel Time (μ s)
Baseline (Naive)	64	256	16×16	84.5	4199899
Baseline (Naive)	32	256	16×16	88.5	4498256
Baseline (Naive)	8	1024	16×16	71.6	6611119
Baseline (Naive)	8	256	16×16	62.5	5077420
Opt1 (Tiled Conv)	64	256	16×16	93.6	4181863
Opt1 (Tiled Conv)	8	512	16×16	67.9	5462704
Opt1 (Tiled Conv)	8	1024	16×16	83.2	6524291
Opt1 (Tiled Conv)	8	256	16×16	62.3	5009579
Opt2 (Fused Conv+Bias+ReLU)	64	256	16×16	83.51	4296950
Opt2 (Fused Conv+Bias+ReLU)	8	512	16×16	71.7	5734764
Opt2 (Fused Conv+Bias+ReLU)	8	1024	16×16	67.17	6558107
Opt2 (Fused Conv+Bias+ReLU)	8	256	16×16	68.0	4978762
Opt2 (Fused Conv+Bias+ReLU)	32	256	16×16	73.1	4475917
Opt2 (Fused Conv+Bias+ReLU)	64	512	16×16	75.1	4235282
Opt2 (Fused) + Tile Change	64	512	8×8	73.6	4196260
Opt2 (Fused) + Tile Change	8	256	8×8	62.9	5002725
Opt3,4 (Streams + Reduction)	64	256	16×16	38.9	428901
Opt3,4 (Streams + Reduction)	8	512	16×16	34.25	1584978
Opt3,4 (Streams + Reduction)	8	1024	16×16	50.7	2519363
Opt3,4 (Streams + Reduction)	8	256	16×16	32.2	1270796
Opt3,4 (Streams + Reduction)	32	256	16×16	46.2	590936
Opt3,4 (Streams + Reduction)	64	512	16×16	30.0	456422

Table 4: Complete profiling results for all configurations. Kernel time represents the sum of CUDA kernel durations reported by Nsight Systems.

7.3 Analysis of Results

7.3.1 Baseline Performance Characteristics

The baseline implementation exhibits several performance characteristics that reveal fundamental bottlenecks:

High Synchronization Overhead: As shown in Nsight Systems profiling, the baseline spends significant time in `cudaDeviceSynchronize()` and `cudaMemcpy()`. This indicates a serialized execution pattern where:

1. CPU prepares batch data
2. CPU transfers data to GPU via `cudaMemcpy()` (GPU idle)
3. GPU executes kernels while CPU waits
4. GPU transfers results back to CPU
5. CPU processes next batch while GPU is idle

This pattern creates substantial idle time on both CPU and GPU, with utilization never reaching 100%.

Batch Size Impact:

- **Batch=64:** Best performance (4.2s GPU time, 84.5% utilization)
- **Batch=32:** Slightly slower (4.5s, 88.5% utilization)
- **Batch=8:** Significantly slower (5.1-6.6s, 62-72% utilization)

Block Size Impact:

- Block=256: Best for most cases
- Block=1024: Worse performance (6.6s vs 5.1s for batch=8)
- Block=512: Intermediate performance

Observation: Larger block sizes do not always improve performance. Block=1024 likely suffers from reduced occupancy due to register/shared memory pressure.

7.3.2 Optimization 1: Tiled Convolution Impact

Performance Gains:

Configuration	Baseline (μs)	Opt1 (μs)
Batch=64, Block=256	4,199,899	4,181,863
Batch=8, Block=256	5,077,420	5,009,579
Batch=8, Block=1024	6,611,119	6,524,291

Table 5: Tiled Convolution Performance Comparison

Key Findings:

- **GPU utilization increased:** From 84.5% to 93.6% for batch=64
- **Better occupancy:** Tiling enables more threads to run concurrently

7.3.3 Optimization 2: Kernel Fusion Impact

Performance Comparison:

Configuration	Opt1 (μs)	Opt2 (μs)	Change
Batch=64, Block=256	4,181,863	4,296,950	-2.8% slower
Batch=8, Block=256	5,009,579	4,978,762	+0.6% faster
Batch=32, Block=256	N/A	4,475,917	N/A

Table 6: Kernel Fusion vs Tiling Alone

Fusion provides benefits:

- Reduced kernel launch overhead
- Fewer intermediate memory operations
- Better for larger networks where launch overhead accumulates

7.3.4 Tile Size Experiment (8×8 vs 16×16)

Results:

Configuration	16×16 (μs)	8×8 (μs)
Batch=64, Block=512	4,235,282	4,196,260
Batch=8, Block=256	4,978,762	5,002,725

Table 7: Tile Size Impact on Performance

Observations:

- Minimal difference between 8×8 and 16×16 tiles
- 16×16 slightly better for batch=64 (1% faster)
- 8×8 slightly better for batch=8

Trade-offs:

- **16×16 tiles:**
 - More data reuse ($16 \times 16 = 256$ elements for 256 outputs)
 - Higher shared memory usage
 - Better for larger batches
- **8×8 tiles:**
 - Less shared memory
 - Higher occupancy possible
 - Better for small batches

7.3.5 Optimization 3+4: Dramatic Improvement

Performance Breakthrough:

Configuration	Opt2 (μ s)	Opt3+4 (μ s)
Batch=64, Block=256	4,296,950	428,901
Batch=32, Block=256	4,475,917	590,936
Batch=8, Block=256	4,978,762	1,270,796

Table 8: Atomic Reduction + Stream Overlapping Impact

Key Insight:

- Kernels execute $10\times$ faster (atomic reduction optimization)
- Total time decreased $4.5\times$ (wall-clock)
- GPU idle time stayed constant (CPU preprocessing bottleneck)
- Utilization appears lower because GPU finishes so quickly it spends more *percentage* of time waiting

Lower utilization with much lower active time means:

- Kernels are extremely efficient
- Next bottleneck is CPU preprocessing
- Further optimization should target CPU side

7.3.6 Breaking Down Optimization 3+4 Contributions

We implemented two optimizations together:

1. **Atomic Reduction (Opt3):** Block-level reduction before atomic operations
2. **Stream Overlapping (Opt4):** Concurrent H2D transfer and GPU execution

Why Better Than Expected?

1. Atomic reduction improves cache behavior (fewer cache misses)
2. Better instruction-level parallelism in optimized backward
3. Compiler can optimize better with reduced atomic contention

Key Takeaways:

1. Small optimizations (tiling, fusion) had minimal impact on our problem size
2. Algorithmic optimization (atomic reduction) provided massive speedup
3. Stream overlapping removed synchronization bottleneck
4. Final implementation is faster than baseline
5. Next bottleneck is CPU preprocessing (not GPU compute)

8 Conclusions

8.1 Project Summary

This project successfully implemented a complete CNN training pipeline in pure CUDA, achieving all primary objectives:

Technical Achievements:

1. **Complete Implementation:** All forward and backward kernels developed from scratch
2. **Correctness Verified:** Network trains successfully, achieving 95-99% accuracy on MNIST
3. **Four Optimizations Implemented:** Tiling, fusion, atomic reduction, and streaming
4. **Significant Speedup:** improvement over baseline implementation
5. **Comprehensive Analysis:** Detailed profiling and performance characterization

8.1.1 About Neural Network Training

1. Backward Pass is Expensive:

Our profiling revealed that backward pass (computing gradients) takes ~50% of training time despite doing "the same" operations as forward pass. This is because:

- Must compute three gradients per layer (input, weights, bias)
- Requires saving all forward activations (memory intensive)
- Memory access patterns are less favorable (transposed operations)

2. Batch Size Has Complex Trade-offs:

Larger batches provide better GPU utilization but:

- Require more memory
- Can hurt convergence (gradient estimates become less noisy)

Our results show batch=64 is optimal for MNIST, but this varies by problem.

8.1.2 About GPU Programming

1. Memory Hierarchy Matters:

Shared memory tiling is a powerful technique that can provide more speedup on memory-bound workloads. However, our results show minimal benefit for MNIST (28×28 images, 3×3 kernels) because:

- L1/L2 caches already capture most reuse for small images
- 3×3 kernels have limited reuse opportunity
- Modern GPUs have large caches that benefit naive implementations

Shared memory tiling becomes critical for larger images (224×224), larger kernels (5×5 , 7×7), or when training very deep networks where memory bandwidth saturates.

9 References

1. <https://www.geeksforgeeks.org/c/restrict-keyword-c/>
2. <https://www.youtube.com/watch?v=PmqHkytaRSU>
3. <https://www.youtube.com/watch?v=w8yWXqWQYmU>
4. <https://www.youtube.com/watch?v=QzY57FaENXg>